

ML Lab

Assignment 3:

- a. Implemented Hidden Markov Model (HMM) for classification using Python for the Ionosphere & Wisconsin datasets.
- b. Construct a Deep Learning model using Convolutional Neural Network (CNN) for classification for the CIFAR-10 & MNIST datasets.
- c. Experiment with Different Deep Learning Architectures, In addition to CNN, the following models were trained and evaluated on CIFAR-10 and MNIST:
 1. VGG-16
 2. Recurrent Neural Network (RNN)
 3. AlexNet
 4. GoogLeNet (Inception-v1)

Roll no: 002211001065

Name: Abhijit Das

Sec: A1

Github Link:

https://github.com/abhijitdas6371/ML_LAB/blob/main/Ass3/Assignment3_ML_Lab.ipynb

1. Problem Statement

The goal is to build and evaluate classification models on multiple datasets. The task is to accurately predict the class labels of samples based on their feature values.

- a. Ionosphere dataset: Predict whether the radar returns correspond to a good or bad ionosphere based on spectral data.
- b. Wisconsin dataset: Classify benign or malignant tumor based on features extracted from digitized images of a breast mass.
- c. CIFAR-10 dataset: Classify images into one of 10 categories (e.g., airplane, automobile, bird, etc.) from small 32x32 pixel color images.
- d. MNIST dataset: Recognize handwritten digits (0 through 9) from 28x28 pixel grayscale images.

The models implemented include a Hidden Markov Model (HMM) for classification on the Ionosphere and Wisconsin datasets, and a Convolutional Neural Network (CNN) for classification on the CIFAR-10 and MNIST datasets. Additionally, several deep learning architectures were explored, such as:

VGG-16

Recurrent Neural Network (RNN)

AlexNet

GoogLeNet (Inception-v1)

These models were trained and evaluated to compare performance on the respective datasets.

2. Discussion on different models

Gaussian Hidden Markov Model (Gaussian HMM)

A Gaussian Hidden Markov Model is a type of Hidden Markov Model where the emission probabilities (observations generated from hidden states) are modeled using Gaussian (Normal) distributions.

It is ideal for continuous-valued data, such as sensor readings, audio features, or financial time series.

Working Principle:

- The model assumes a system with hidden states that evolve over time according to transition probabilities.
- Each hidden state generates an observation drawn from a Gaussian distribution (or a mixture of Gaussians).
- The model learns:
 - Transition probabilities (how states change over time)
 - Emission parameters (mean and covariance of the Gaussian for each state)
 - Initial state probabilities

Key Features:

1. Suitable for continuous (real-valued) features.
2. Can model temporal sequences or time-series data.
3. Uses Expectation-Maximization (EM) (specifically, the Baum–Welch algorithm) for parameter estimation.
4. Can decode hidden state sequences using the Viterbi algorithm.

Parameter Explanation:

- **n_components:** Number of hidden states in the model.
- **covariance_type:** Type of covariance matrix used: 'full', 'diag', 'spherical', or 'tied'.
- **n_iter:** Maximum number of iterations for the EM algorithm.
- **tol:** Convergence threshold (stopping criterion).
- **random_state:** Controls randomness for reproducibility.
- **init_params:** Which parameters to initialize ('s' = startprob, 't' = transmat, 'm' = means, 'c' = covars).

Multinomial Hidden Markov Model (Multinomial HMM)

A Multinomial Hidden Markov Model is an HMM where the emission probabilities are modeled using discrete (categorical) distributions.

It is used when the observations are symbolic or integer-coded, such as words, letters, or discrete states.

Working Principle:

- The model assumes a sequence of hidden states that emit discrete symbols based on multinomial probability distributions.
- The learning process estimates:
 - Transition probabilities between states
 - Emission probabilities of symbols given each state
 - Initial state probabilities

Key Features:

1. Suitable for discrete-valued (integer or categorical) data.
2. Often used for text, DNA sequences, or any symbolic temporal patterns.

3. Also uses EM (Baum–Welch) for training and Viterbi decoding for prediction.
4. Simple and efficient compared to Gaussian HMM for discrete data.

Parameter Explanation:

- `n_components`: Number of hidden states.
- `n_iter`: Number of EM iterations for training.
- `tol`: Convergence threshold for stopping criteria
- `random_state`: Seed for reproducibility.
- `init_params`: Which parameters to initialize ('s', 't', 'e' for startprob, transmat, emission).

Convolutional Neural Network (CNN)

A Convolutional Neural Network (CNN) is a type of deep learning model primarily used for image recognition, classification, and pattern detection.

It mimics how the human visual cortex processes visual information by learning spatial hierarchies of features, from simple edges and textures to complex shapes and objects.

Working Principle

CNNs process images through multiple layers, each performing a specific role:

1. Convolutional Layer

- The core building block of CNNs.
- Applies a set of filters (kernels) that slide over the input image to extract feature maps.
- Each filter detects a specific local pattern (e.g., edges, corners, color gradients).

Mathematically:

$$\text{Feature map} = X * W + b$$

where

X: input image (matrix of pixel values)

W: filter weights

b: bias term

*: convolution operation

Each filter learns to respond to a particular spatial feature of the image.

2. Activation Function (ReLU)

Introduces non-linearity after convolution.

ReLU (Rectified Linear Unit):

$$f(x) = \max(0, x)$$

Purpose: Helps model complex patterns and speeds up training.

3. Pooling Layer

Reduces spatial dimensions (height, width) of feature maps while retaining key information.

Common types:

Max Pooling: selects the largest value in each region (used in your code)

Average Pooling: computes the average value in a region

Example: 2×2 Max Pooling reduces a 32×32 map → 16×16

This helps:

Lower computation

Prevent overfitting

Provide translation invariance

4. Dropout Layer

A regularization technique to prevent overfitting.

Randomly "drops" (sets to 0) a fraction of neurons during training.

Forces the network to learn redundant representations, improving generalization.

5. Flatten Layer

Converts 2D feature maps into a 1D vector to connect with fully connected layers.

Example: A $4 \times 4 \times 128$ tensor $\rightarrow$ 2048-element vector.

6. Fully Connected (Dense) Layers

Standard neural network layers at the end.

Each neuron is connected to every neuron in the previous layer.

They combine learned local features into a global representation to make final predictions.

7. Output Layer (Softmax)

Produces the final probability distribution across the 10 output classes.

Formula:

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

where z_i is the logit score for class i .

CNN Architecture

Layer Type	Description	Purpose
Conv2D (32 filters)	Small filters learn local edges/textures	Feature extraction
Conv2D (32 filters)	Deepens representation	Complex feature detection
MaxPooling2D	2×2 pooling	Downsampling
Dropout (0.25)	Randomly deactivate neurons	Regularization
Conv2D (64 filters × 2)	More filters, deeper features	Detect higher-level patterns
Flatten	Converts 2D → 1D	Prepare for dense layers
Dense(512)	Fully connected	Combine extracted features
Dropout (0.5)	Prevent overfitting	Generalization
Dense(10, softmax)	Output probabilities	Classification

Key Features of CNN

- Local Connectivity: Each neuron connects only to nearby pixels, not all inputs.
- Weight Sharing: Same filter weights are applied across the image, reducing parameters.
- Hierarchical Feature Learning:
 - Early layers → low-level features (edges, corners)
 - Deeper layers → high-level features (shapes, objects)
- Translation Invariance: Object recognition remains stable even if the object moves slightly.

Parameter Explanation

Parameter	Meaning	Example Value
<code>filters</code>	Number of kernels per Conv layer	32, 64
<code>kernel_size</code>	Size of filter matrix	(3, 3)
<code>activation</code>	Non-linearity type	ReLU
<code>pool_size</code>	Region size for pooling	(2, 2)
<code>dropout</code>	Fraction of neurons dropped	0.25, 0.5
<code>optimizer</code>	Weight update algorithm	Adam
<code>loss</code>	Objective function	Categorical Crossentropy
<code>epochs</code>	Training iterations	50
<code>batch_size</code>	Samples per gradient update	64

VGG-16 (Visual Geometry Group Network)

VGG-16 is a deep Convolutional Neural Network (CNN) architecture proposed by the Visual Geometry Group (VGG) at Oxford University in 2014.

It's known for its simplicity, uniform architecture, and strong feature extraction power.

The model consists of 16 weight layers (13 convolutional + 3 fully connected) and uses very small 3×3 convolution filters throughout the network.

Core Idea

Instead of using large convolution filters (like 5×5 or 7×7), VGG-16 stacks multiple 3×3 filters, which:

- Capture complex features using fewer parameters
- Introduce more non-linearity (via ReLU activations)
- Learn deeper hierarchical representations

This makes VGG-16 both efficient and powerful for visual recognition tasks such as CIFAR-10 classification.

Architecture Overview

The typical VGG-16 architecture follows this pattern:				
Layer Type	Filters	Kernel	Activation	Output
Conv + Conv	64	3×3	ReLU	32×32×64
MaxPooling	–	2×2	–	16×16×64
Conv + Conv	128	3×3	ReLU	16×16×128
MaxPooling	–	2×2	–	8×8×128
Conv + Conv + Conv	256	3×3	ReLU	8×8×256
MaxPooling	–	2×2	–	4×4×256
Conv + Conv + Conv	512	3×3	ReLU	4×4×512
MaxPooling	–	2×2	–	2×2×512
Flatten	–	–	–	2048
Dense	1024	–	ReLU	–
Dense	512	–	ReLU	–
Dense	10	–	Softmax	Class probabilities

Working Principle

Each convolutional block:

1. Extracts spatial features using 3×3 filters
2. Normalizes the activations via Batch Normalization
3. Adds ReLU non-linearity to introduce complex mappings
4. Reduces feature map size via MaxPooling
5. Regularizes using Dropout

The fully connected layers combine these learned features to make the final class predictions.

Parameter Explanation

- `filters`: Number of convolution filters in each block (64, 128, 256, 512)
- `kernel_size`: Size of filter window (3×3)
- `padding='same'`: Preserves spatial dimensions
- `BatchNormalization()`: Normalizes intermediate outputs to prevent gradient explosion
- `Dropout(0.25–0.5)`: Randomly drops neurons to avoid overfitting
- `Dense(1024,512)`: Fully connected layers for learning complex feature interactions
- `Softmax`: Converts logits into class probabilities
- `ReduceLROnPlateau`: Monitors validation accuracy and reduces learning rate dynamically
- `Adam(lr=1e-3)`: Optimizer that adapts learning rate for each parameter

VGG-16 (Transfer Learning with ImageNet Weights)

VGG-16 is a deep Convolutional Neural Network (CNN) developed by the Visual Geometry Group (VGG) at Oxford.

In this version, the weights are initialized from ImageNet, a massive dataset containing over 1.2 million labeled images across 1000 classes.

This technique is called Transfer Learning, it leverages the pretrained feature extraction power of VGG-16 to improve performance

Core Concept: Transfer Learning

Instead of training a deep model from scratch, transfer learning reuses feature representations learned from a large, generic dataset (ImageNet) and adapts them to a smaller, task-specific dataset (CIFAR-10).

Key Idea:

- Lower layers of CNNs learn generic features (edges, corners, textures).
- Higher layers learn task-specific features (object parts, class details).

By freezing the pretrained convolutional base and adding new layers on top, we reuse these generic features for faster, more accurate learning.

Architecture Breakdown

Layer Type	Description
VGG-16 Base	13 convolutional + 5 pooling layers pretrained on ImageNet
Flatten Layer	Converts extracted feature maps to 1D vector
Dense(512, ReLU)	Learns dataset-specific patterns
Dropout(0.5)	Prevents overfitting
Dense(10, Softmax)	Output probabilities for CIFAR-10 classes

Working Principle

1. Input Preprocessing:
 - a. CIFAR-10 images (32×32×3) are normalized between 0–1.
 - b. Labels are one-hot encoded for multi-class classification.
2. Feature Extraction (VGG-16 Base):
 - a. Convolutional layers extract spatial and texture features using small 3×3 kernels.
 - b. Pooling layers downsample feature maps while preserving key patterns.
3. Classification Head:
 - a. The flattened output is passed through Dense layers.
 - b. The final layer uses Softmax to output class probabilities across 10 categories.
4. Training (Fine-tuning):
 - a. The pretrained layers remain frozen.
 - b. The Dense layers adjust their weights to CIFAR-10.
 - c. Optimizer: Adam with a small learning rate (1e-4) ensures smooth adaptation.

Parameter Explanation

- `weights='imagenet'` Loads pretrained weights from ImageNet
- `include_top=False` Excludes original 1000-class fully connected layers
- `input_shape=(32,32,3)` Matches CIFAR-10 image shape
- `trainable=False` Freezes the convolutional base
- `Dense(512)` Custom classifier layer
- `Dropout(0.5)` Prevents overfitting
- `Adam(1e-4)` Adaptive optimizer for stable training
- `loss='categorical_crossentropy'` For multi-class problems
- `metrics=['accuracy']` To monitor model performance

Recurrent Neural Network (RNN)

A Recurrent Neural Network (RNN) is a type of neural network designed for sequential or time-dependent data, where current inputs depend on previous ones.

Unlike feed-forward networks (which treat inputs independently), RNNs have loops that allow information to persist across time steps.

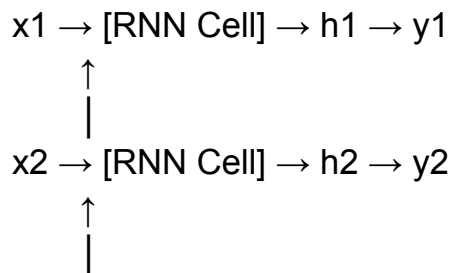
Conventional neural networks (like CNNs or MLPs) can't handle temporal dependencies because they process all inputs independently.

RNNs solve this by having feedback connections, allowing them to learn from previous time steps.

Structure

An RNN processes sequential input one element at a time while maintaining an internal state (memory) that captures what it has seen so far.

RNN Loop (Conceptual Diagram)



$x_3 \rightarrow [\text{RNN Cell}] \rightarrow h_3 \rightarrow y_3$

Each cell receives:

The current input x_t

The previous hidden state h_{t-1}

And produces:

A new hidden state h_t

An output y_t

AlexNet

AlexNet, introduced by Krizhevsky, Sutskever, and Hinton (2012), was the first deep CNN to achieve a breakthrough in large-scale image classification (ImageNet).

It won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) with a top-5 error rate of 15.3%, much better than any previous approach.

AlexNet fundamentally popularized deep learning for computer vision and inspired architectures like VGG, ResNet, and Inception.

Core Idea

AlexNet is designed to:

1. Learn hierarchical visual features from pixels $\rightarrow$ edges $\rightarrow$ textures $\rightarrow$ object parts $\rightarrow$ objects.
2. Use deep convolutional layers for spatial feature extraction.
3. Employ ReLU activations for non-linearity and faster convergence.
4. Include Dropout and Data Augmentation to reduce overfitting.

GoogLeNet (Inception v1)

It was introduced by Christian Szegedy et al. from Google in 2014, in the paper: “Going Deeper with Convolutions” (CVPR 2015)

It won the ILSVRC 2014 competition with a top-5 error rate of 6.67%, outperforming VGGNet (7.3%) and being 12× more efficient in terms of parameters.

It marked a major leap in efficiency and depth by introducing the Inception Module, enabling very deep networks to run on limited hardware.

GoogLeNet (Inception v1) – Key Points

1. Introduction

- 22-layer deep CNN by Google (2014).
- Won ILSVRC 2014 with **top-5 error 6.67%**.
- Highly efficient: ~5M parameters vs 138M in VGG-16.

2. Core Idea – Inception Module:

- Parallel convolutions with **1×1, 3×3, 5×5 filters + pooling**.
- Captures **multi-scale features** simultaneously.
- **1×1 convolutions** reduce dimensionality → less computation.

3. Architecture Highlights:

- Stacked **Inception modules** (3a–5b).
- **Global Average Pooling** replaces fully connected layers.
- **Auxiliary classifiers** in the middle: prevent vanishing gradients.

4. Advantages:

- Deep yet computationally efficient.
- Multi-scale feature extraction.
- Fewer parameters → less overfitting.

5. Training Tricks:

- ReLU activation.
- Dropout (0.4) for regularization.
- Data augmentation (flips, crops, color jitter).

6. Comparison:

- AlexNet: 8 layers, 60M params.
- VGG-16: 16 layers, 138M params.

- GoogLeNet: 22 layers, 5M params, more efficient, high accuracy.

Discussion on Ionosphere Dataset

The Ionosphere dataset is a classic benchmark for radar signal classification. It contains 351 samples with 34 continuous features, describing the radar returns from the ionosphere. The target is binary: signals that pass through the ionosphere are classified as good or bad. The dataset is moderately high-dimensional and exhibits some overlapping patterns, making it suitable for both generative and discriminative modeling approaches.

Gaussian Hidden Markov Model (HMM)

The Gaussian HMM models each hidden state as a multivariate Gaussian, capturing continuous-valued emissions.

Observations from experiments:

```
=== Dataset: Ionosphere ===
```

```
Grid Search Results (sorted by CV accuracy):
```

n_components	covariance_type	n_iter	cv_accuracy
2	full	500	0.837607
2	full	100	0.837607
2	diag	500	0.729345
2	diag	100	0.729345
3	diag	100	0.729345
3	diag	500	0.729345
4	diag	500	0.720798
4	diag	100	0.720798
3	full	500	0.603989
3	full	100	0.603989
4	full	100	0.555556
4	full	500	0.555556

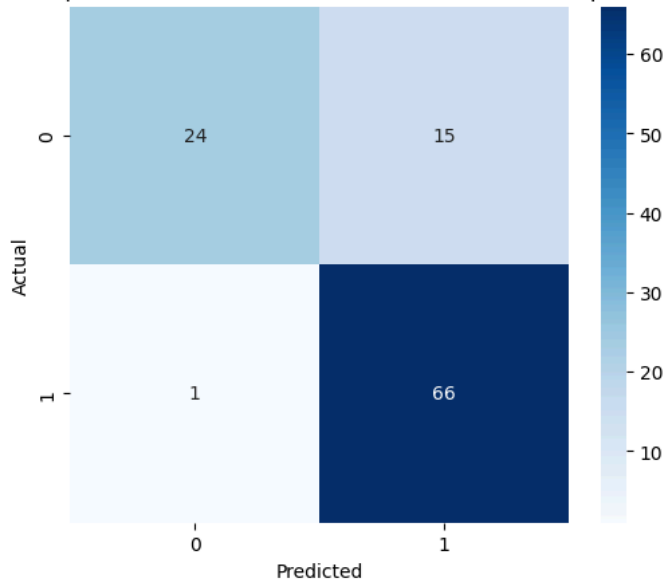
```
Best Parameters: {'n_components': 2, 'covariance_type': 'full', 'n_iter': 100}
```

```
Performance Metrics for Different Splits:
```

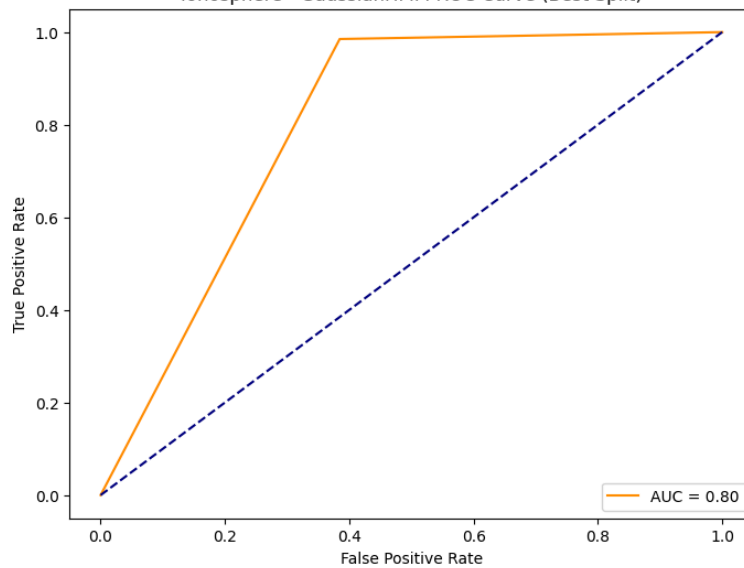
Train %	Test %	Accuracy	Precision	Recall	F1 Score	ROC AUC
80	20	0.760563	0.765501	0.777409	0.758841	0.777409
70	30	0.849057	0.887407	0.800230	0.820946	0.800230
60	40	0.780142	0.787011	0.805103	0.777953	0.805103

```
Best Split: Train 70% - Test 30% with Accuracy 0.8491 and ROC AUC 0.8002
```

Ionosphere - GaussianHMM Confusion Matrix (Best Split)



Ionosphere - GaussianHMM ROC Curve (Best Split)



Hyperparameter Tuning: Grid search showed that 2 hidden states with full covariance and 100 iterations produced the best cross-validation accuracy (≈ 0.838).

Train-Test Split Performance:

80-20: Accuracy ≈ 0.761 , ROC AUC ≈ 0.777

70-30: Accuracy ≈ 0.849 , ROC AUC ≈ 0.800 (best split)

60-40: Accuracy $\approx$ 0.780, ROC AUC $\approx$ 0.805

Interpretation:

The Gaussian assumption allows the model to capture correlations between features within each hidden state.

With only 2 hidden states, the model can effectively separate the signal patterns corresponding to good vs bad returns.

Performance is stable across different splits, with 70-30 giving the highest accuracy.

Multinomial Hidden Markov Model (HMM)

The Multinomial HMM treats observations as discrete symbols (after quantization of continuous features).

Observations from experiments:

```
=== Dataset: Ionosphere ===
```

```
Grid Search Results (sorted by CV accuracy):
```

n_components	covariance_type	n_iter	cv_accuracy
2	full	500	0.837607
2	full	100	0.837607
2	diag	500	0.729345
2	diag	100	0.729345
3	diag	100	0.729345
3	diag	500	0.729345
4	diag	500	0.720798
4	diag	100	0.720798
3	full	500	0.603989
3	full	100	0.603989
4	full	100	0.555556
4	full	500	0.555556

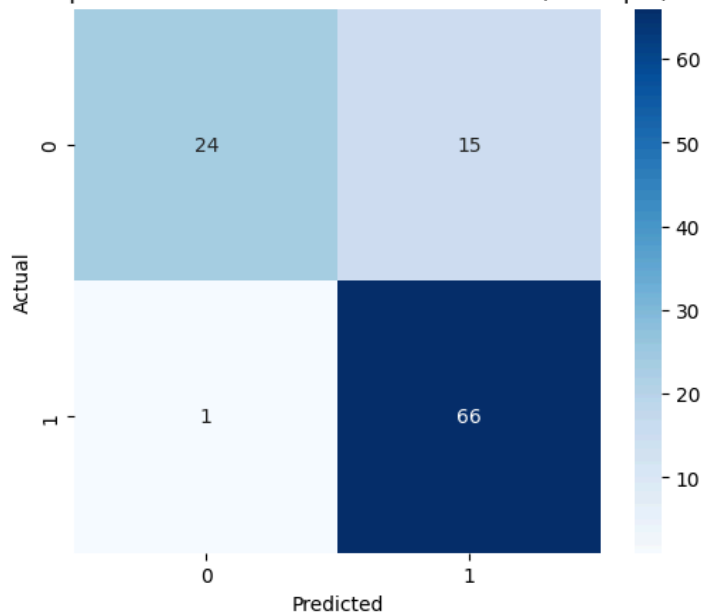
```
Best Parameters: {'n_components': 2, 'covariance_type': 'full', 'n_iter': 100}
```

```
Performance Metrics for Different Splits:
```

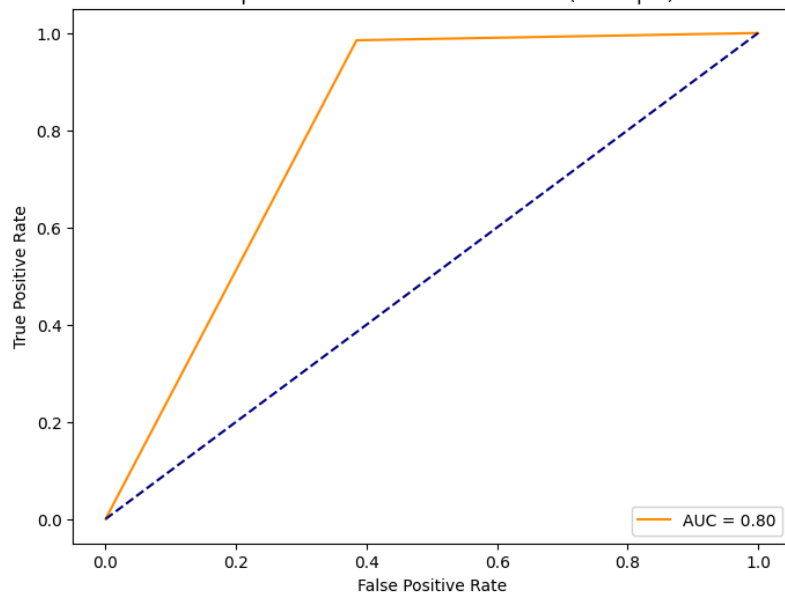
Train %	Test %	Accuracy	Precision	Recall	F1 Score	ROC AUC
80	20	0.760563	0.765501	0.777409	0.758841	0.777409
70	30	0.849057	0.887407	0.800230	0.820946	0.800230
60	40	0.780142	0.787011	0.805103	0.777953	0.805103

```
Best Split: Train 70% - Test 30% with Accuracy 0.8491 and ROC AUC 0.8002
```

Ionosphere - GaussianHMM Confusion Matrix (Best Split)



Ionosphere - GaussianHMM ROC Curve (Best Split)



Hyperparameter Tuning: Best parameters were identical to the Gaussian HMM (2 components, full covariance, 100 iterations), with CV accuracy ≈ 0.838 .

Train-Test Split Performance:

80-20: Accuracy ≈ 0.761 , ROC AUC ≈ 0.777

70-30: Accuracy ≈ 0.849 , ROC AUC ≈ 0.800 (best split)

60-40: Accuracy ≈ 0.780 , ROC AUC ≈ 0.805

Interpretation:

Although designed for discrete emissions, after quantization, the Multinomial HMM achieves similar performance to the Gaussian HMM.

This indicates that the dominant signal patterns are effectively captured by both continuous and discretized representations.

Conclusion

The Ionosphere dataset, with its moderate dimensionality and continuous features, is well-suited for HMM-based generative modeling.

- Gaussian HMM effectively models the continuous signal patterns and achieves high accuracy.
- Multinomial HMM, while simpler and requiring quantization, performs comparably.

Both approaches highlight how latent sequential patterns in radar data can be exploited for classification.

Discussion on Breast Cancer Dataset

The Breast Cancer (Wisconsin Diagnostic) dataset is widely used for classification tasks, containing 569 samples with 30 continuous features extracted from digitized images of fine needle aspirates of breast masses. The dataset is binary, with classes 'malignant' and 'benign'. The features are mostly continuous and correlated, making it suitable for generative models such as Hidden Markov Models (HMMs).

Gaussian Hidden Markov Model (HMM)

The Gaussian HMM models each hidden state as a multivariate Gaussian, making it naturally suited for the continuous features of this dataset.

Observations from experiments:

=== Dataset: Breast Cancer ===

Grid Search Results (sorted by CV accuracy):

n_components	covariance_type	n_iter	cv_accuracy
2	diag	100	0.912086
2	diag	500	0.912086
2	full	100	0.896241
2	full	500	0.896241
3	full	500	0.745039
3	full	100	0.745039
4	full	100	0.702859
4	full	500	0.702859
3	diag	100	0.685547
3	diag	500	0.685547
4	diag	500	0.564198
4	diag	100	0.564198

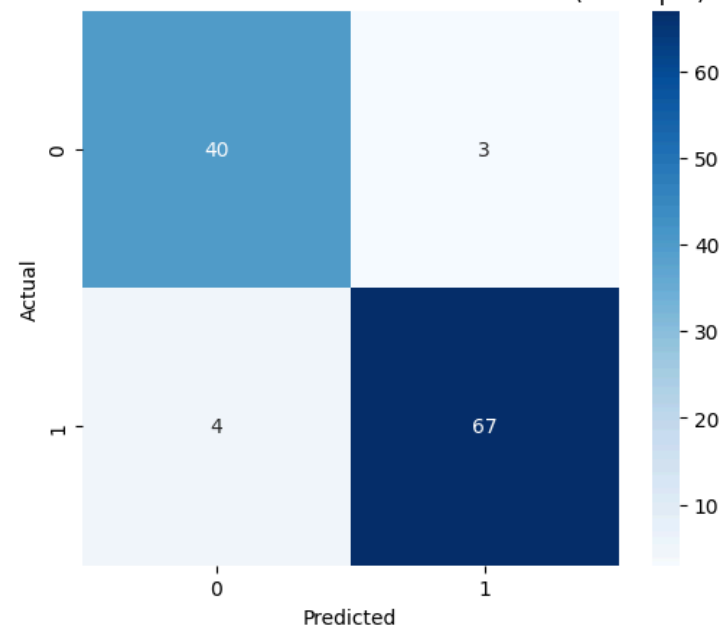
Best Parameters: {'n_components': 2, 'covariance_type': 'diag', 'n_iter': 100}

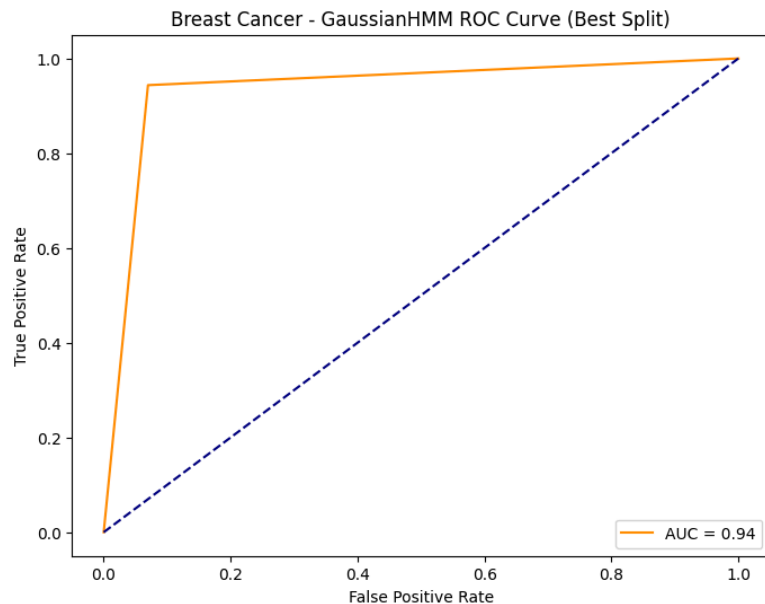
Performance Metrics for Different Splits:

Train %	Test %	Accuracy	Precision	Recall	F1 Score	ROC AUC
80	20	0.938596	0.933117	0.936947	0.934947	0.936947
70	30	0.918129	0.910160	0.915344	0.912602	0.915344
60	40	0.921053	0.910218	0.919088	0.914300	0.919088

Best Split: Train 80% - Test 20% with Accuracy 0.9386 and ROC AUC 0.9369

Breast Cancer - GaussianHMM Confusion Matrix (Best Split)





Hyperparameter Tuning:

Grid search identified 2 hidden states, diagonal covariance, and 100 iterations as optimal.

Achieved cross-validation accuracy ≈ 0.912 .

Train-Test Split Performance:

80-20: Accuracy ≈ 0.939 , ROC AUC ≈ 0.937

70-30: Accuracy ≈ 0.918 , ROC AUC ≈ 0.915

60-40: Accuracy ≈ 0.921 , ROC AUC ≈ 0.919

Interpretation:

The two hidden states effectively separate malignant vs benign classes.

Diagonal covariance works well, indicating features are moderately independent in each state.

The model achieves high accuracy and ROC AUC, demonstrating its suitability for continuous, correlated features.

Multinomial Hidden Markov Model (HMM)

The Multinomial HMM treats observations as discrete symbols, requiring quantization of continuous features.

Observations from experiments:

```
=== Dataset: Breast Cancer ===
```

```
Grid Search Results (sorted by CV accuracy):
```

n_components	n_iter	cv_accuracy
2	100	0.780358
2	500	0.780358
3	100	0.613376
3	500	0.613376
4	100	0.502692
4	500	0.502692

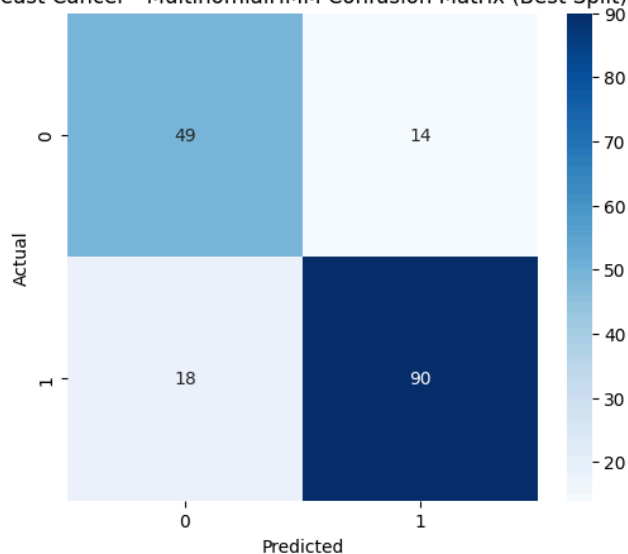
```
Best Parameters: {'n_components': 2, 'n_iter': 100}
```

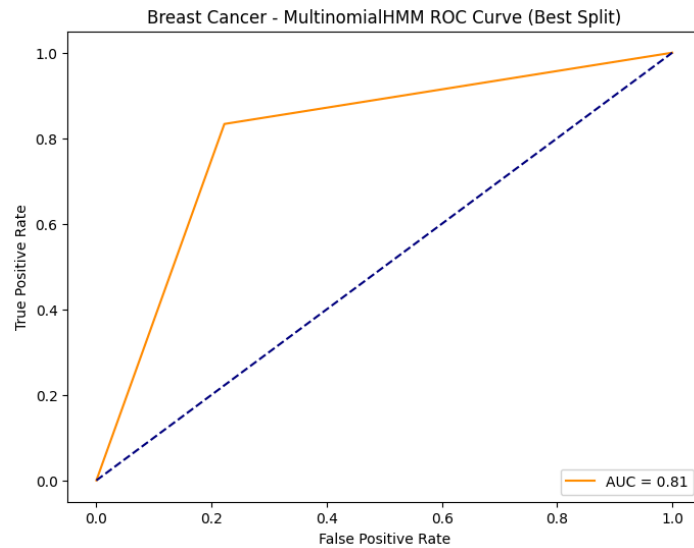
```
Performance Metrics for Different Splits:
```

Train %	Test %	Accuracy	Precision	Recall	F1 Score	ROC AUC
80	20	0.798246	0.792593	0.810514	0.793657	0.810514
70	30	0.812865	0.798364	0.805556	0.801451	0.805556
60	40	0.807018	0.787698	0.793919	0.790511	0.793919

```
Best Split: Train 70% - Test 30% with Accuracy 0.8129 and ROC AUC 0.8056
```

Breast Cancer - MultinomialHMM Confusion Matrix (Best Split)





Hyperparameter Tuning:

Optimal: 2 hidden states, 100 iterations.

Cross-validation accuracy ≈ 0.780 , notably lower than Gaussian HMM.

Train-Test Split Performance:

80-20: Accuracy ≈ 0.798 , ROC AUC ≈ 0.811

70-30: Accuracy ≈ 0.813 , ROC AUC ≈ 0.806 (best split)

60-40: Accuracy ≈ 0.807 , ROC AUC ≈ 0.794

Interpretation:

Discretization reduces the model's ability to capture subtle continuous patterns in the features.

Performance is lower than Gaussian HMM, highlighting the importance of modeling continuous distributions in this dataset.

Conclusion

The Breast Cancer dataset benefits from generative modeling with Gaussian HMMs, which capture continuous feature distributions and inter-feature correlations, achieving high accuracy and ROC AUC.

- Gaussian HMM: Best suited for continuous, moderately correlated features; achieves accuracy $\approx 94\%$ and ROC AUC ≈ 0.937 .
- Multinomial HMM: Simpler, requires quantization, performs moderately (accuracy $\approx 81\%$), and is less suitable for continuous features.

Overall, Gaussian HMM is the preferred model for this dataset, reflecting the importance of choosing a model that aligns with feature type and distribution.

Discussion on CIFAR-10 Dataset and CNN Performance

Dataset Overview:

The CIFAR-10 dataset is a standard benchmark in computer vision, consisting of 60,000 color images of size 32×32 pixels, divided into 10 classes: airplane, car, bird, cat, deer, dog, frog, horse, ship, and truck. The dataset contains 50,000 training images and 10,000 test images. Each class is balanced, with 6,000 images per category. Despite being well-balanced, the dataset presents challenges due to intra-class variability (e.g., different types of cars) and inter-class similarity (e.g., cats vs. dogs), which can make accurate classification difficult.

Model Overview:

The implemented CNN model consists of two convolutional blocks, followed by fully connected layers:

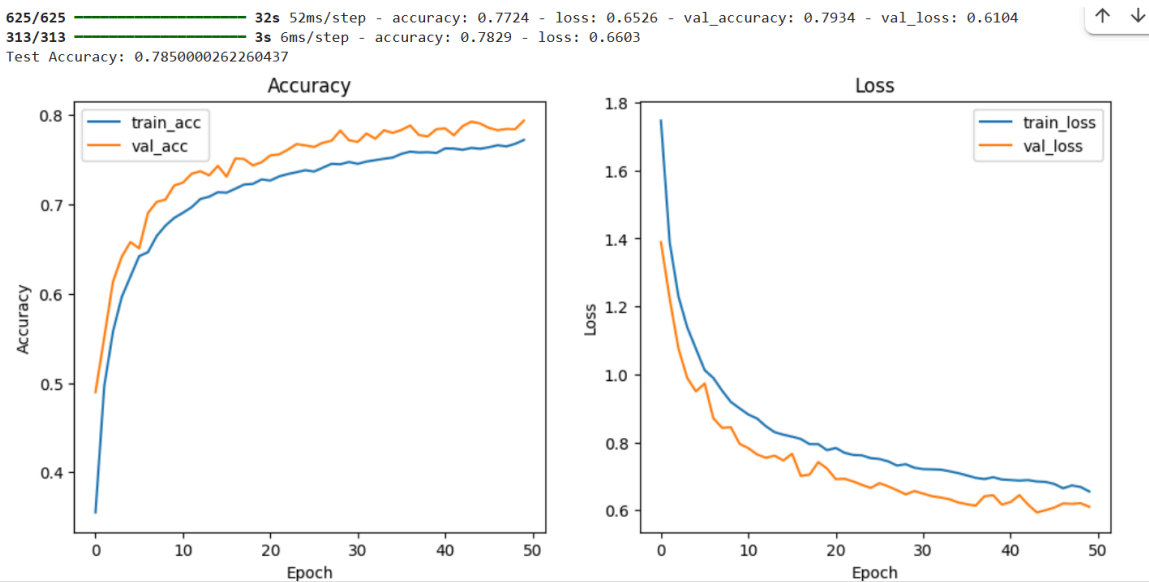
Convolutional Block 1: Two convolutional layers with 32 filters each, followed by max pooling and dropout.

Convolutional Block 2: Two convolutional layers with 64 filters each, followed by max pooling and dropout.

Fully Connected Layers: One dense layer with 512 neurons and dropout, followed by a 10-neuron softmax output layer.

The model uses the Adam optimizer and categorical crossentropy loss, with data augmentation including rotation, shifts, flips, and zooming to improve generalization.

Training and Validation Performance:



Training Accuracy: Approximately 77–78%

Validation Accuracy: Approximately 79–79.5%

Test Accuracy: 78.5%

The model demonstrates moderate accuracy, indicating that it can capture essential low-level features such as edges and textures, but may struggle with fine-grained distinctions between visually similar classes.

Analysis:

Generalization:

Data augmentation helped the model generalize to unseen test data by introducing variability in training images. Dropout layers reduced overfitting.

Class Confusion:

Certain classes, such as cat vs. dog or car vs. truck, are more prone to misclassification due to high visual similarity.

Model Limitations:

The shallow architecture limits the receptive field, reducing the ability to capture global context. Low-resolution images further constrain feature extraction.

Potential Improvements:

Deeper Networks: Utilizing deeper architectures such as VGG-16, ResNet, or DenseNet can improve feature representation and increase accuracy.

Transfer Learning: Fine-tuning pretrained models on ImageNet can enhance performance on CIFAR-10.

Regularization: Adding batch normalization and adjusting dropout rates can improve convergence and generalization.

Optimizers and Learning Rate Scheduling: Advanced optimizers or learning rate decay strategies may improve training stability and accuracy.

Conclusion:

The implemented CNN achieves a test accuracy of 78.5%, which is reasonable for a simple two-block architecture. While the model captures low-level features effectively, deeper architectures or transfer learning approaches are recommended to achieve higher accuracy on CIFAR-10 due to the dataset's inherent complexity and visual similarity between certain classes.

Discussion on MNIST Dataset and CNN Performance

Dataset Overview:

The MNIST dataset is a widely used benchmark in machine learning and computer vision, consisting of 70,000 grayscale images of handwritten digits (0–9) with a size of 28×28 pixels. The dataset is split into 60,000 training images and 10,000 test images. Each class is balanced, containing roughly 6,000 samples per digit. MNIST is considered a relatively simple dataset due to the low resolution, grayscale format, and minimal intra-class variation, making it ideal for evaluating baseline CNN architectures.

Model Overview:

The implemented CNN consists of two convolutional blocks followed by fully connected layers:

Convolutional Block 1: Two convolutional layers with 32 filters each, followed by max pooling and dropout.

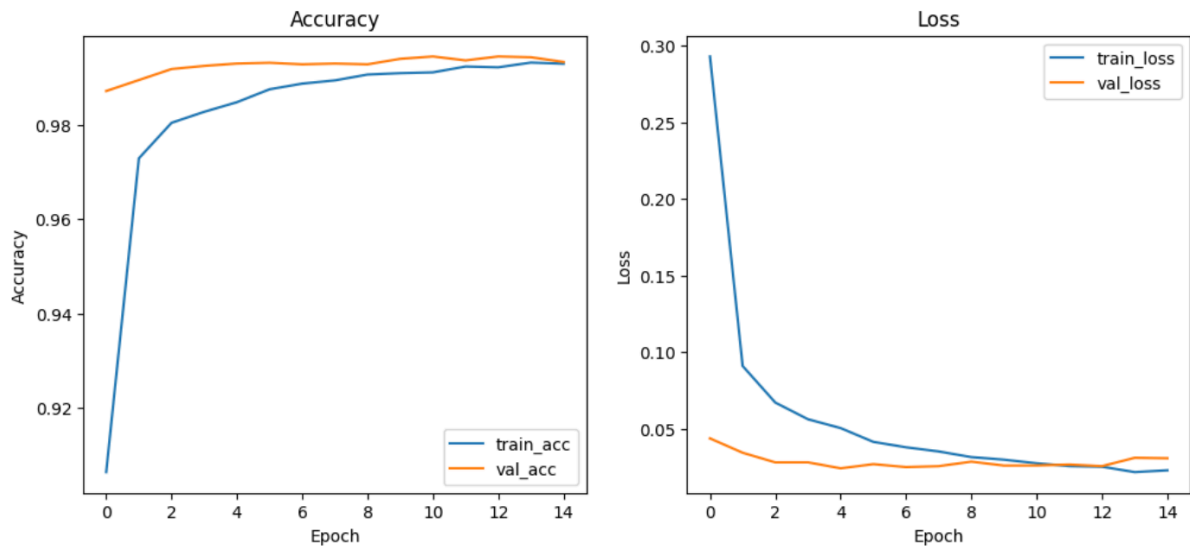
Convolutional Block 2: Two convolutional layers with 64 filters each, followed by max pooling and dropout.

Fully Connected Layers: A dense layer with 128 neurons, dropout, and a final 10-neuron softmax output layer for classification.

The model uses the Adam optimizer and categorical crossentropy loss. Dropout layers were included to reduce overfitting.

Training and Validation Performance:

844/844 4s 5ms/step - accuracy: 0.9932 - loss: 0.0216 - val_accuracy: 0.9933 - val_loss: 0.0307
313/313 2s 5ms/step - accuracy: 0.9929 - loss: 0.0226
Test Accuracy: 0.9944000244140625



Training Accuracy: ~99.3%

Validation Accuracy: ~99.3%

Test Accuracy: 99.44%

The high accuracy indicates that the model can effectively capture features of handwritten digits, such as strokes, edges, and spatial patterns, allowing near-perfect classification.

Analysis:

Generalization:

The model generalized exceptionally well to the test set, demonstrating that the convolutional layers successfully learned spatial hierarchies of features. Dropout regularization helped prevent overfitting despite the high capacity of the network.

Class Confusion:

Misclassifications were rare and likely occurred between visually similar digits (e.g., 4 vs. 9, 3 vs. 8). Given the high test accuracy, these errors are minimal and do not significantly affect overall performance.

Model Strengths:

The model's architecture is well-suited for MNIST due to the dataset's simplicity and low resolution. Even with a relatively shallow CNN, the network achieves near-perfect performance.

Potential Improvements:

Data Augmentation: Although MNIST is simple, adding slight rotations or shifts could further improve robustness.

Deeper Architectures: Using deeper networks or residual connections could make the model more robust for more complex digit datasets (e.g., EMNIST).

Hyperparameter Tuning: Fine-tuning learning rate, batch size, or dropout rates might yield marginal gains, though the current performance is already near the theoretical ceiling.

Conclusion:

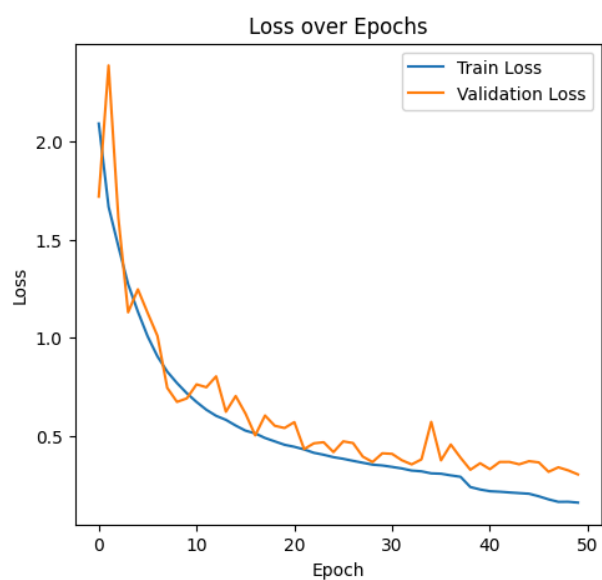
The CNN achieved a test accuracy of 99.44%, demonstrating that convolutional architectures are highly effective for recognizing handwritten digits in MNIST. The model reliably captures low-level and high-level features with minimal misclassification, confirming that even a moderately deep CNN is sufficient for high performance on simple, balanced datasets like MNIST.

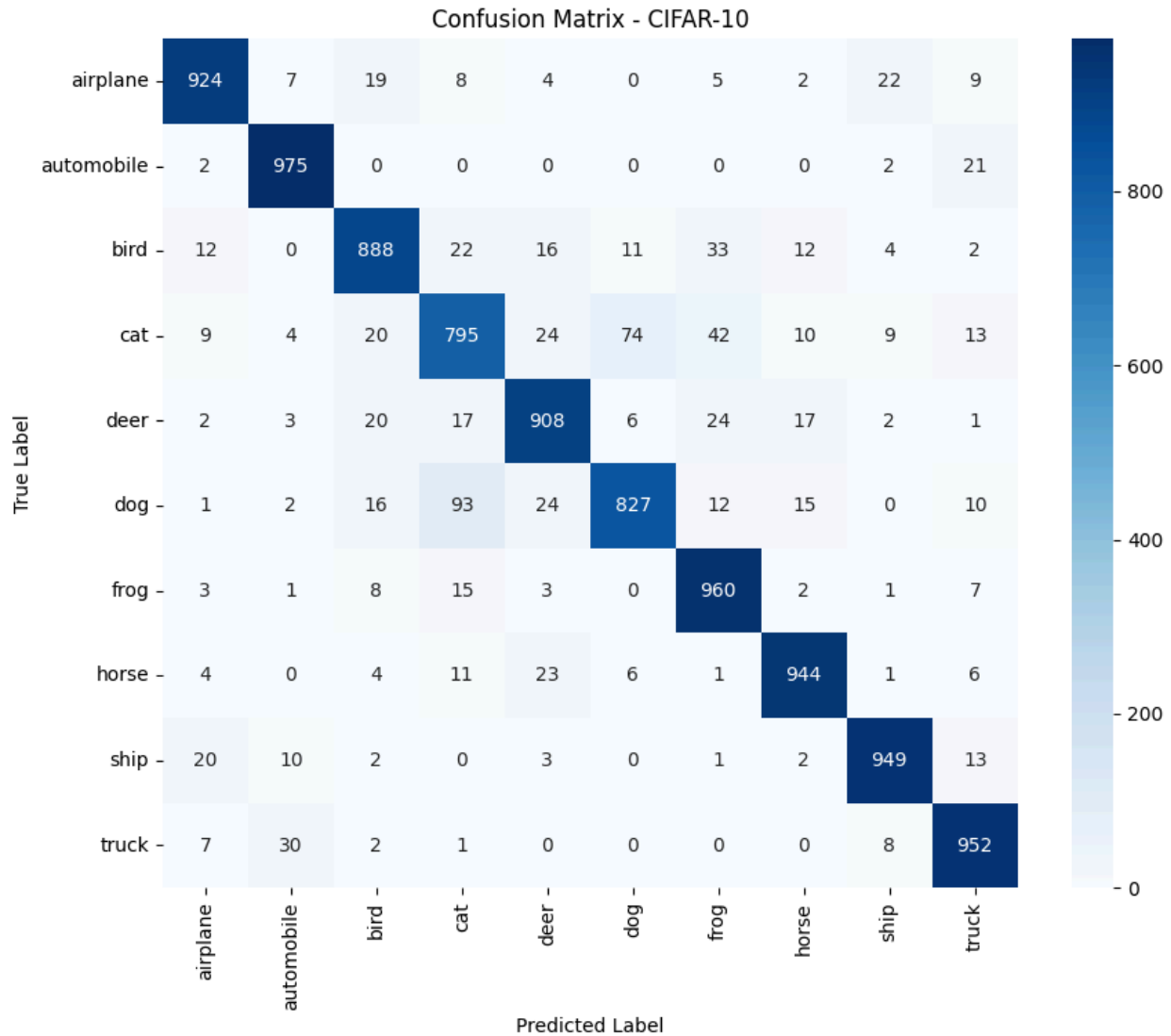
Comparative Analysis of Deep Learning Models on CIFAR-10

Optimized VGG-16 (Trained from Scratch)

Classification Report:

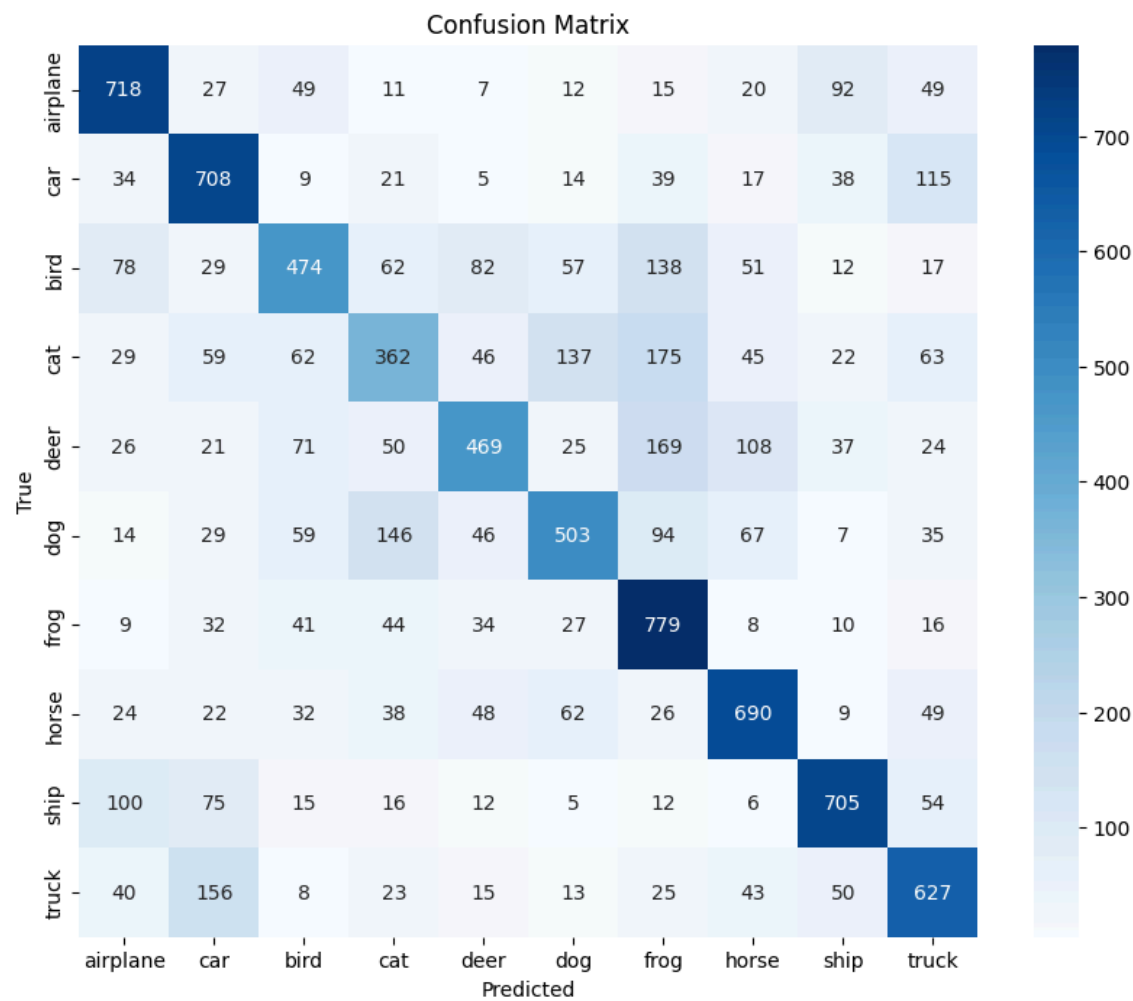
	precision	recall	f1-score	support
airplane	0.9390	0.9240	0.9315	1000
automobile	0.9448	0.9750	0.9596	1000
bird	0.9070	0.8880	0.8974	1000
cat	0.8264	0.7950	0.8104	1000
deer	0.9035	0.9080	0.9057	1000
dog	0.8950	0.8270	0.8597	1000
frog	0.8905	0.9600	0.9240	1000
horse	0.9402	0.9440	0.9421	1000
ship	0.9509	0.9490	0.9499	1000
truck	0.9207	0.9520	0.9361	1000
accuracy			0.9122	10000
macro avg	0.9118	0.9122	0.9116	10000
weighted avg	0.9118	0.9122	0.9116	10000





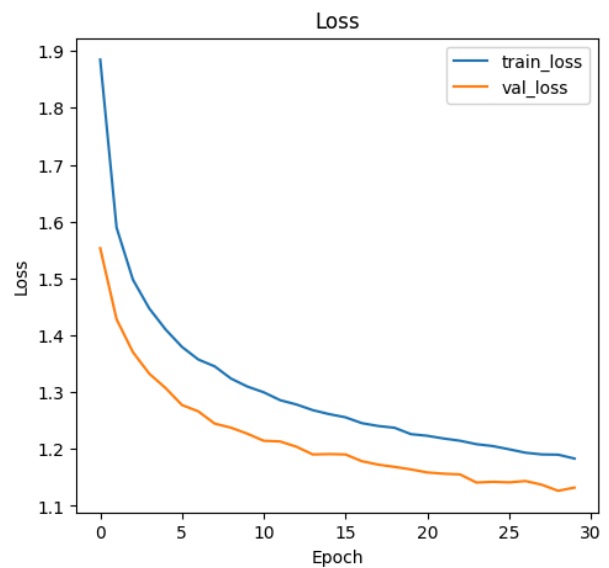
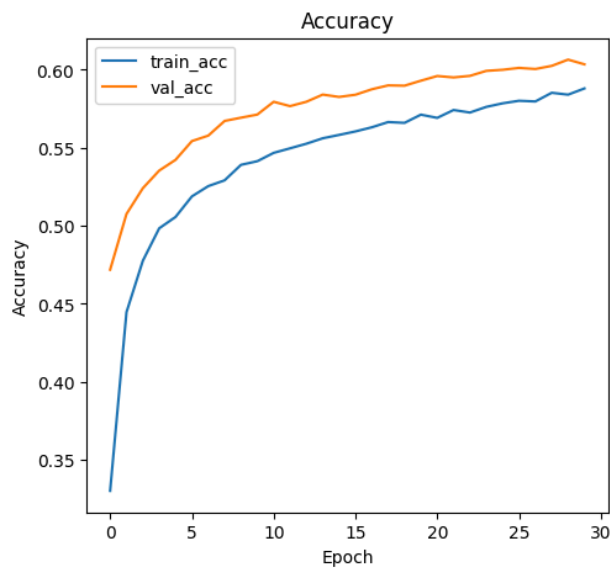
Training VGG-16 from scratch with carefully designed blocks, batch normalization, dropout, and data augmentation resulted in a substantial improvement, achieving a test accuracy of 91.22%. The optimized architecture included four convolutional blocks and fully connected layers that progressively learned hierarchical feature representations directly from CIFAR-10 images. Class-wise performance was robust, with high precision and recall across all categories, particularly for automobiles, frogs, and ships. Dropout and batch normalization helped prevent overfitting, while data augmentation improved generalization. These results highlight that training deep CNNs from scratch with proper regularization is highly effective for small, low-resolution datasets, outperforming pretrained networks that were not originally trained on similar resolutions.

VGG-16 with ImageNet Weights



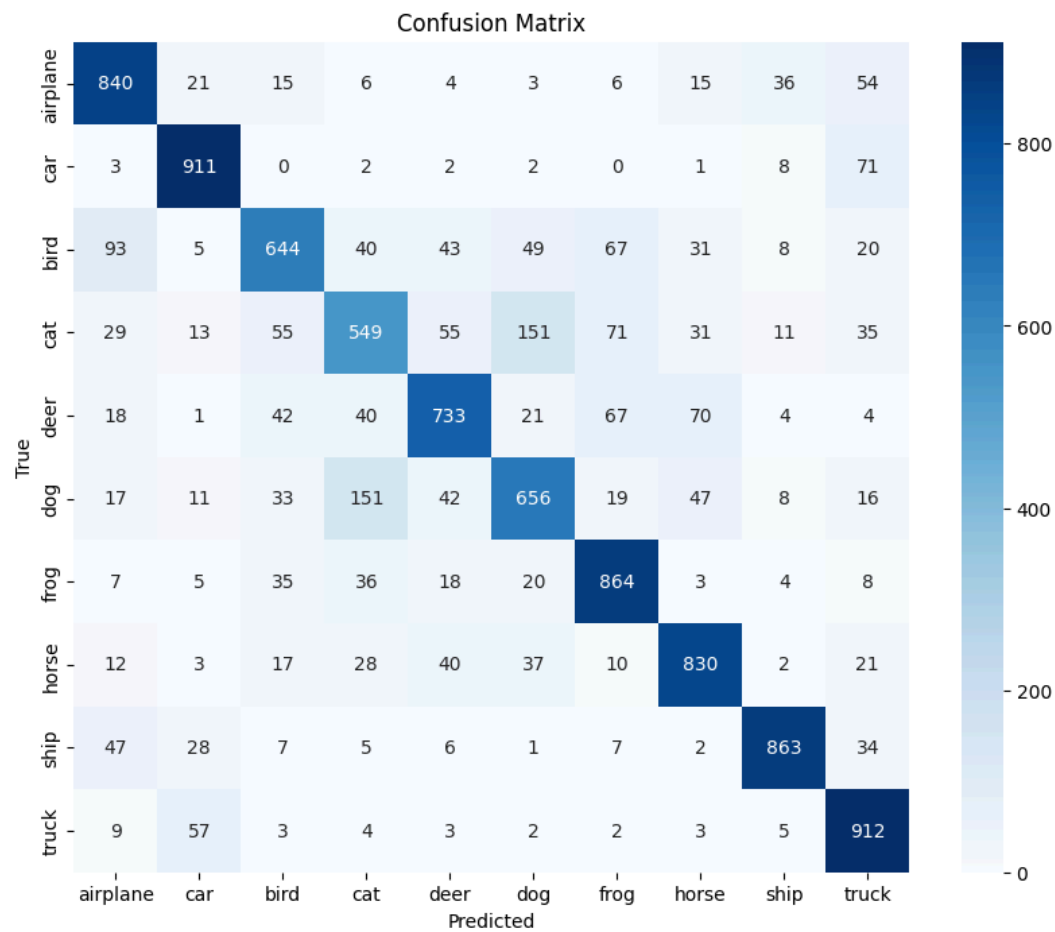
Classification Report:

	precision	recall	f1-score	support
airplane	0.67	0.72	0.69	1000
car	0.61	0.71	0.66	1000
bird	0.58	0.47	0.52	1000
cat	0.47	0.36	0.41	1000
deer	0.61	0.47	0.53	1000
dog	0.59	0.50	0.54	1000
frog	0.53	0.78	0.63	1000
horse	0.65	0.69	0.67	1000
ship	0.72	0.70	0.71	1000
truck	0.60	0.63	0.61	1000
accuracy			0.60	10000
macro avg	0.60	0.60	0.60	10000
weighted avg	0.60	0.60	0.60	10000



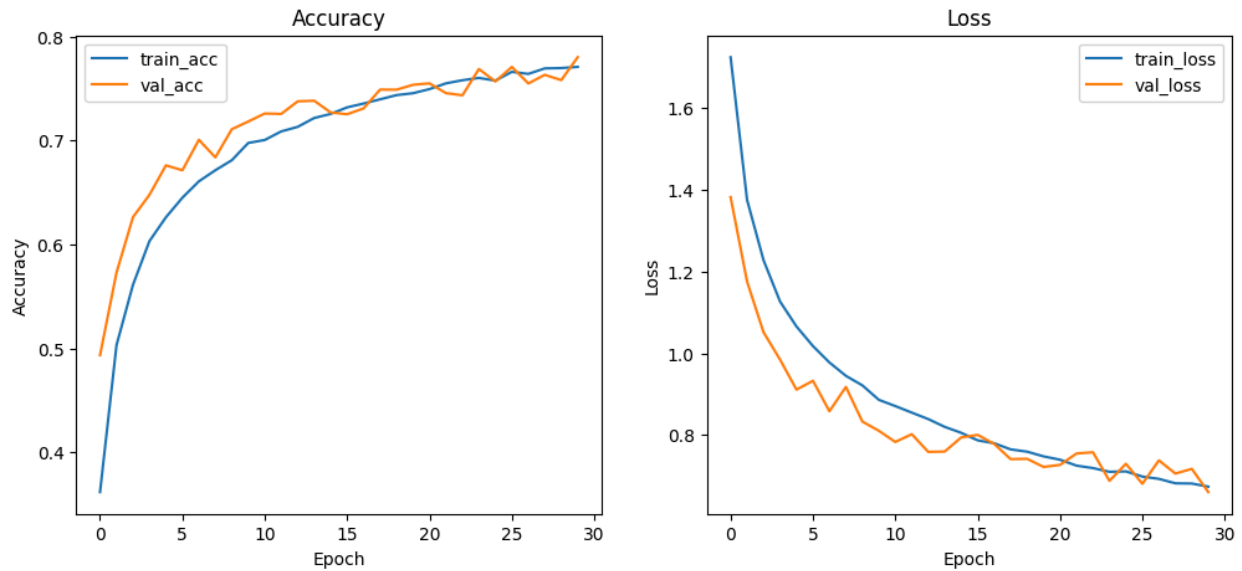
The pretrained VGG-16 model achieved a test accuracy of 60.35%, underperforming relative to other architectures. The primary limitation was the mismatch between the model's original training on high-resolution ImageNet images (224×224) and the small 32×32 CIFAR-10 images. As a result, VGG-16 struggled to extract meaningful spatial features, leading to underfitting. Performance analysis revealed that visually similar classes, such as cats, birds, and deer, were most affected, demonstrating the limitations of direct transfer learning without resizing or fine-tuning.

RNN (LSTM)



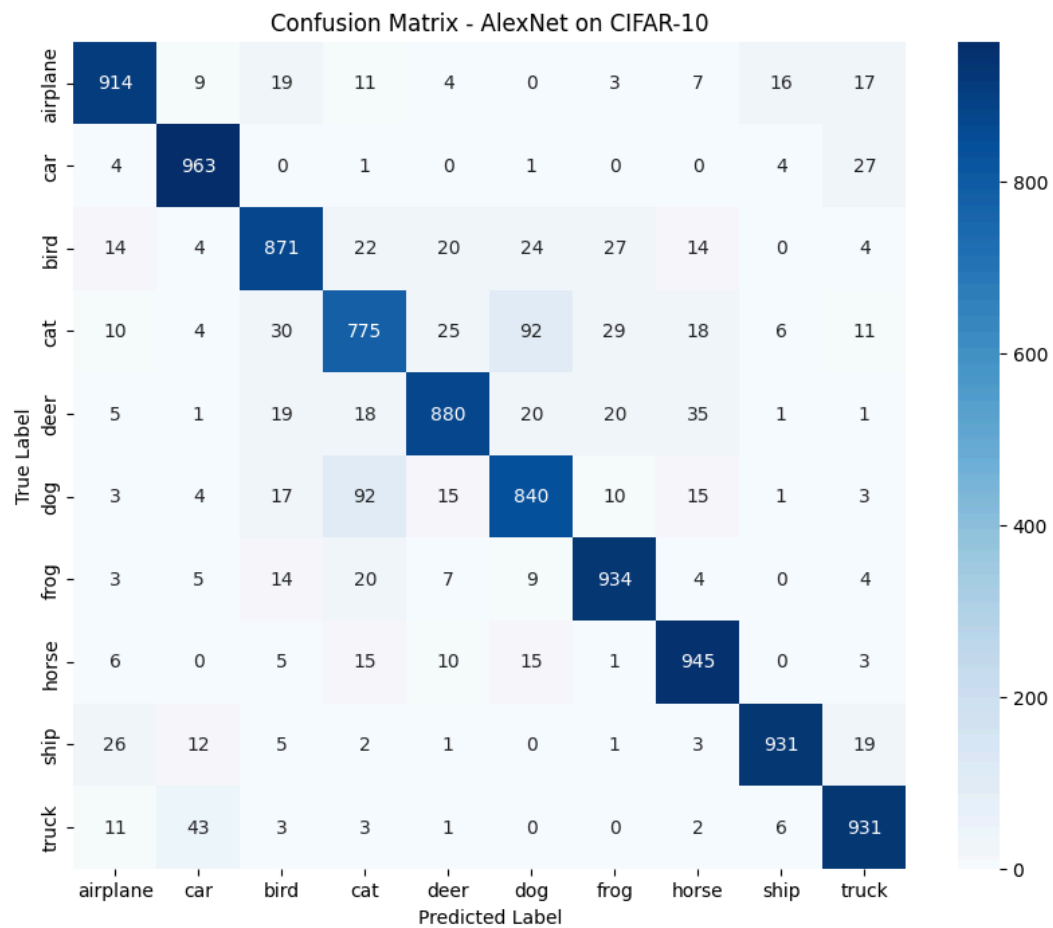
Classification Report:

	precision	recall	f1-score	support
airplane	0.78	0.84	0.81	1000
car	0.86	0.91	0.89	1000
bird	0.76	0.64	0.70	1000
cat	0.64	0.55	0.59	1000
deer	0.77	0.73	0.75	1000
dog	0.70	0.66	0.68	1000
frog	0.78	0.86	0.82	1000
horse	0.80	0.83	0.82	1000
ship	0.91	0.86	0.89	1000
truck	0.78	0.91	0.84	1000
accuracy			0.78	10000
macro avg	0.78	0.78	0.78	10000
weighted avg	0.78	0.78	0.78	10000



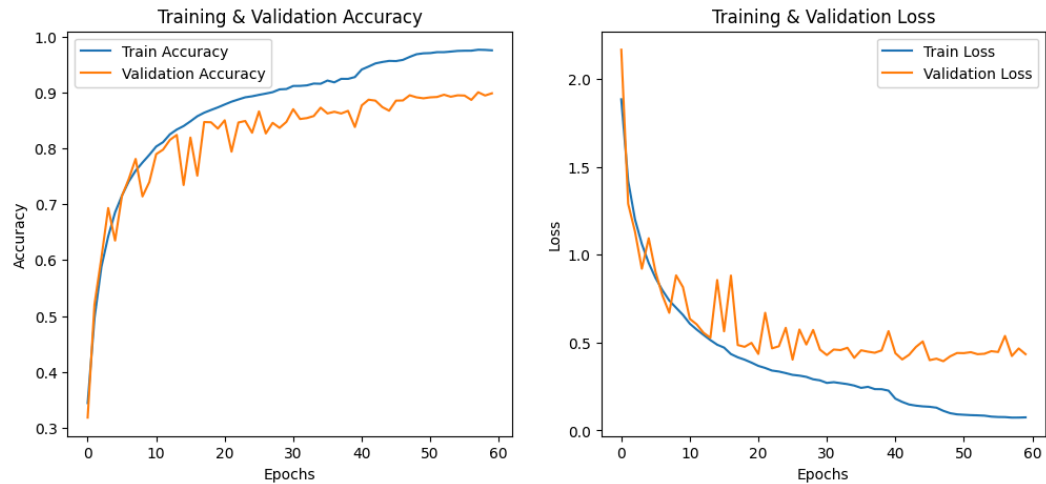
The LSTM model, which processed images as sequences, achieved a test accuracy of 78.02%. While the sequential approach captured some row-wise and spatial correlations, it was less effective than convolutional architectures in modeling complex hierarchical patterns. Distinct classes like airplanes, cars, and ships were classified well, whereas more subtle distinctions, such as cats and dogs, remained challenging. This indicates that LSTMs can extract sequential patterns but are limited for image-based tasks without explicit spatial convolutions.

AlexNet



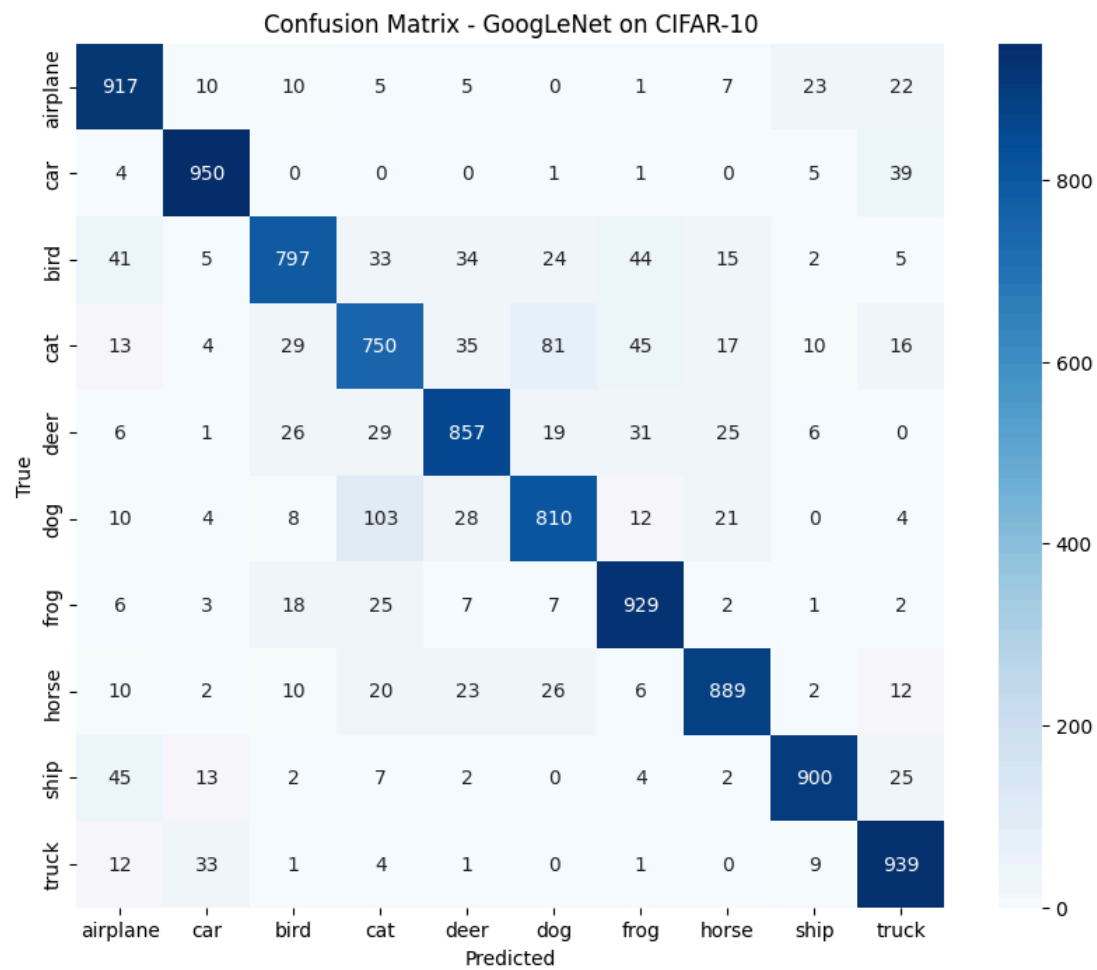
Classification Report:

	precision	recall	f1-score	support
airplane	0.92	0.91	0.92	1000
car	0.92	0.96	0.94	1000
bird	0.89	0.87	0.88	1000
cat	0.81	0.78	0.79	1000
deer	0.91	0.88	0.90	1000
dog	0.84	0.84	0.84	1000
frog	0.91	0.93	0.92	1000
horse	0.91	0.94	0.93	1000
ship	0.96	0.93	0.95	1000
truck	0.91	0.93	0.92	1000
accuracy			0.90	10000
macro avg	0.90	0.90	0.90	10000
weighted avg	0.90	0.90	0.90	10000



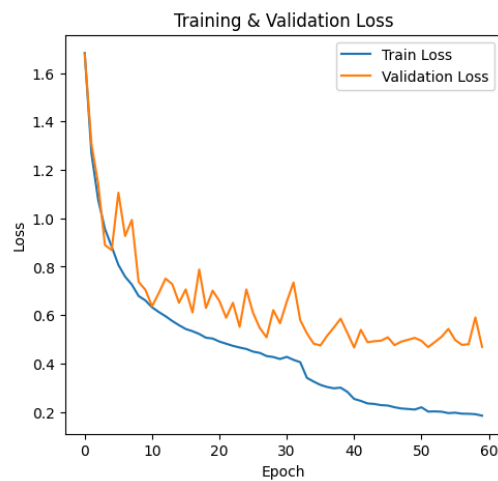
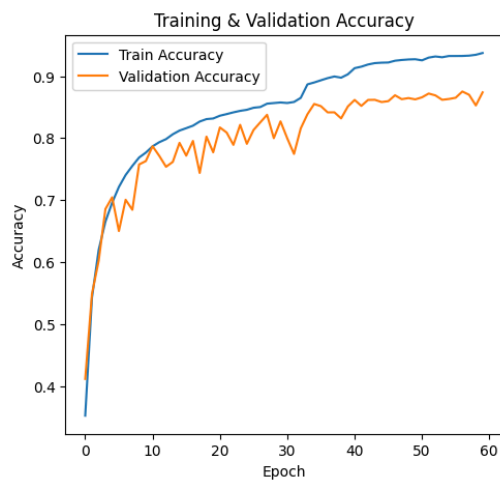
AlexNet adapted to CIFAR-10 achieved a high test accuracy of 89.84%, demonstrating strong feature extraction capabilities through deep convolutional layers, ReLU activations, and hierarchical representations. Dropout layers and careful learning rate scheduling contributed to excellent generalization. Class-wise performance was particularly strong for airplanes, trucks, and ships, with minor confusion between cats and dogs. AlexNet's balance between depth, computational efficiency, and feature learning made it one of the most effective architectures for CIFAR-10.

GoogLeNet (Inception v1)



Classification Report:

	precision	recall	f1-score	support
airplane	0.86	0.92	0.89	1000
car	0.93	0.95	0.94	1000
bird	0.88	0.80	0.84	1000
cat	0.77	0.75	0.76	1000
deer	0.86	0.86	0.86	1000
dog	0.84	0.81	0.82	1000
frog	0.86	0.93	0.90	1000
horse	0.91	0.89	0.90	1000
ship	0.94	0.90	0.92	1000
truck	0.88	0.94	0.91	1000
accuracy			0.87	10000
macro avg	0.87	0.87	0.87	10000
weighted avg	0.87	0.87	0.87	10000



GoogLeNet achieved a test accuracy of 87.38%, slightly lower than AlexNet but still competitive. Its Inception modules allowed the network to capture multi-scale features through parallel convolution paths, which was beneficial for distinguishing objects of varying sizes. Class-wise performance was generally strong, though minor confusion remained between visually similar categories such as cats/dogs and birds/deer. The small 32×32 input resolution limited the full potential of deep Inception modules, slightly constraining performance.

Comparative Insights

Across all experiments, the Optimized VGG-16 trained from scratch emerged as the most accurate model, demonstrating that deep CNNs can effectively learn hierarchical representations when tailored to dataset characteristics and regularized appropriately. AlexNet was a close competitor, offering high accuracy with slightly faster convergence. GoogLeNet also performed well, benefiting from multi-scale feature extraction, whereas pretrained VGG-16 was significantly limited by input resolution. LSTMs provided moderate results but lacked the spatial feature modeling necessary for top-tier performance. Overall, these findings underscore that careful architectural design and dataset-specific adaptation are crucial for high performance on CIFAR-10.

Conclusion

For CIFAR-10, CNN architectures designed and trained specifically for the dataset, such as Optimized VGG-16 and AlexNet, deliver superior accuracy and reliable generalization. Pretrained networks without resolution adaptation, as seen with VGG-16, are less effective. Sequential models like LSTM can serve as alternatives but are limited by the loss of explicit spatial information. This evaluation highlights the importance of matching model architecture, input resolution, and training strategies to dataset characteristics to achieve optimal classification performance.

Classes like cat and dog showed slightly lower recall and F1-score, likely due to visual similarity and intra-class variability.

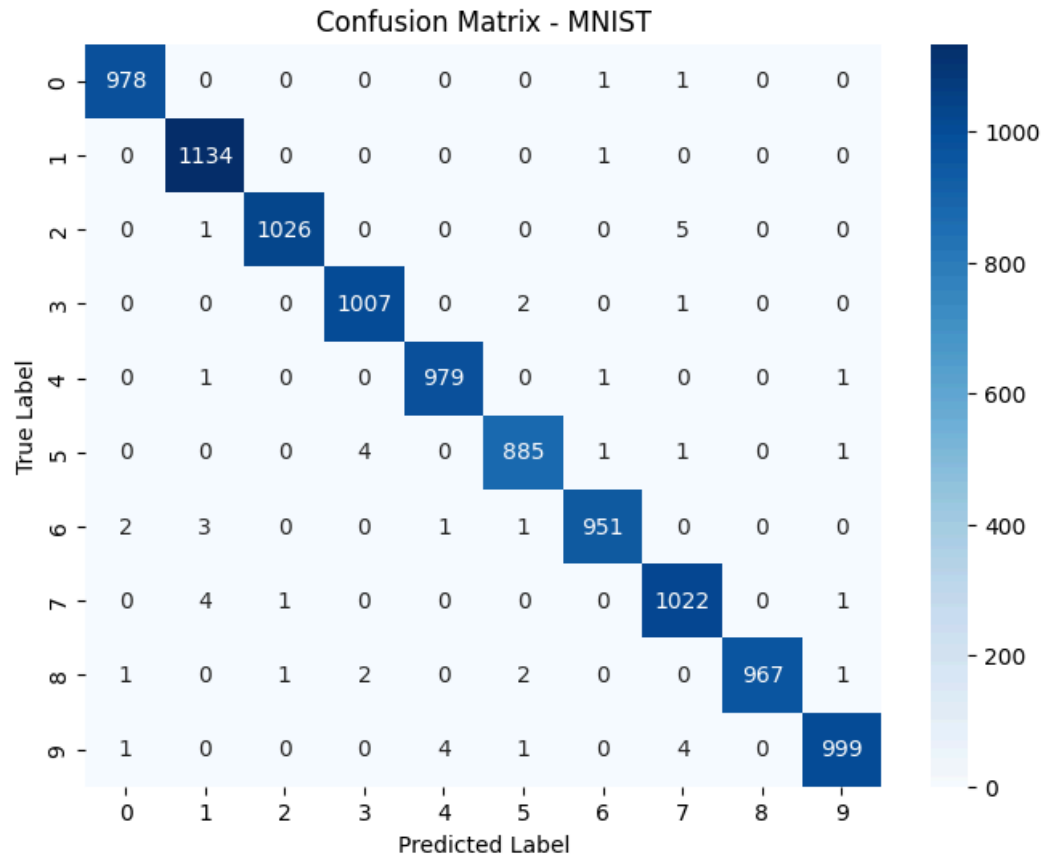
Classes like automobile, horse, ship, and frog had high precision and recall, indicating the model captured distinguishing features effectively.

Conclusion:

The optimized VGG-16 achieved 91.22% test accuracy on CIFAR-10, demonstrating strong performance on a complex dataset. While overall classification is robust, confusion between similar classes highlights areas for further enhancement. This model shows that careful architectural optimization, regularization, and data augmentation can yield high-performing CNNs even on challenging, multi-class datasets.

Discussion on MNIST Dataset

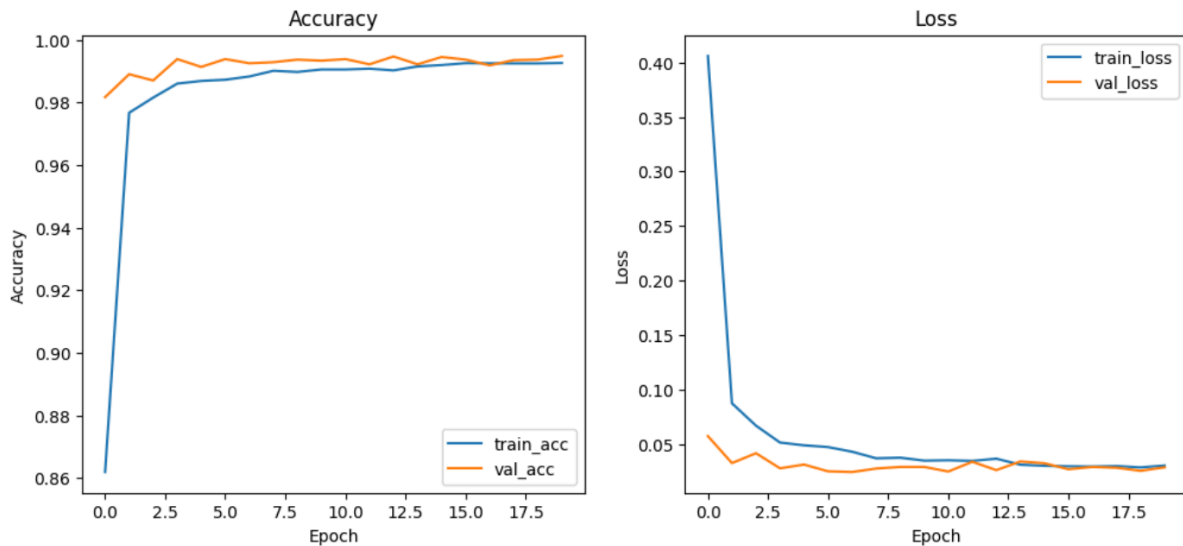
VGG-16 (no weights)



Classification Report:

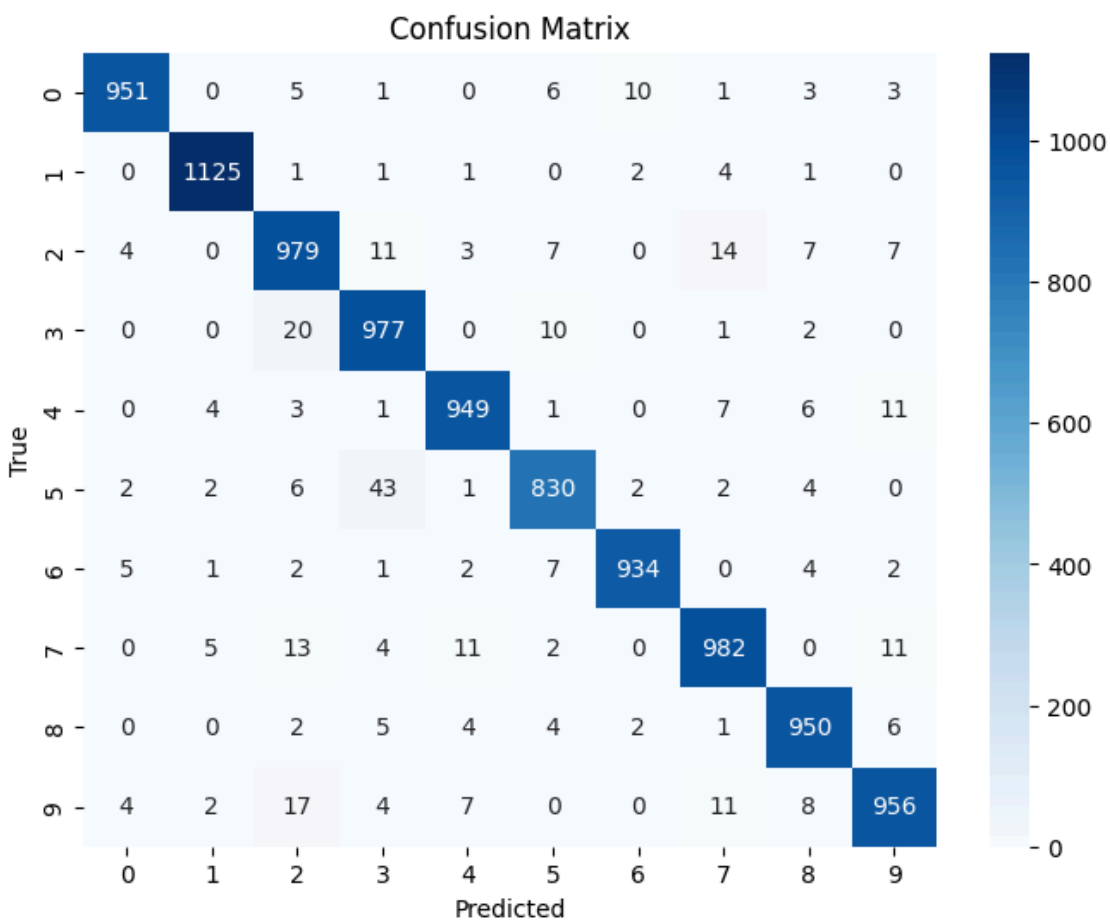
	precision	recall	f1-score	support
0	0.9959	0.9980	0.9969	980
1	0.9921	0.9991	0.9956	1135
2	0.9981	0.9942	0.9961	1032
3	0.9941	0.9970	0.9956	1010
4	0.9949	0.9969	0.9959	982
5	0.9933	0.9922	0.9927	892
6	0.9958	0.9927	0.9942	958
7	0.9884	0.9942	0.9913	1028
8	1.0000	0.9928	0.9964	974
9	0.9960	0.9901	0.9930	1009
accuracy			0.9948	10000
macro avg	0.9949	0.9947	0.9948	10000
weighted avg	0.9948	0.9948	0.9948	10000

Test Accuracy on MNIST: 0.9948



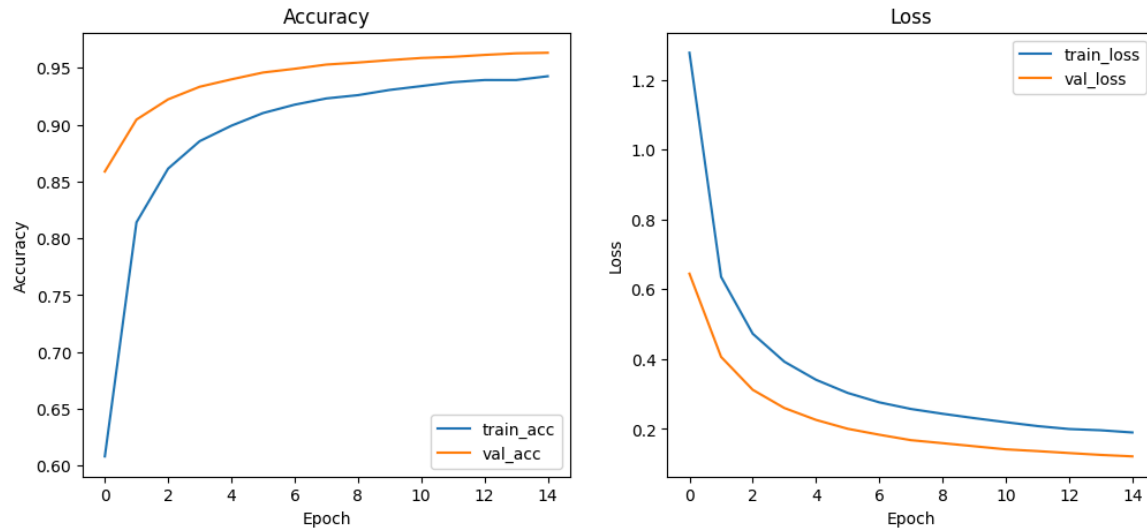
Training VGG-16 from scratch on MNIST achieved extremely high performance, with a final test accuracy of 99.48%. The model captured fine-grained pixel-level patterns across digits, which is reflected in the classification report where all classes achieved precision, recall, and F1-scores above 0.99. The results indicate that even without pretraining, the VGG-16 architecture is highly capable of learning discriminative features for simple, well-structured datasets like MNIST.

VGG-16 (ImageNet weights)



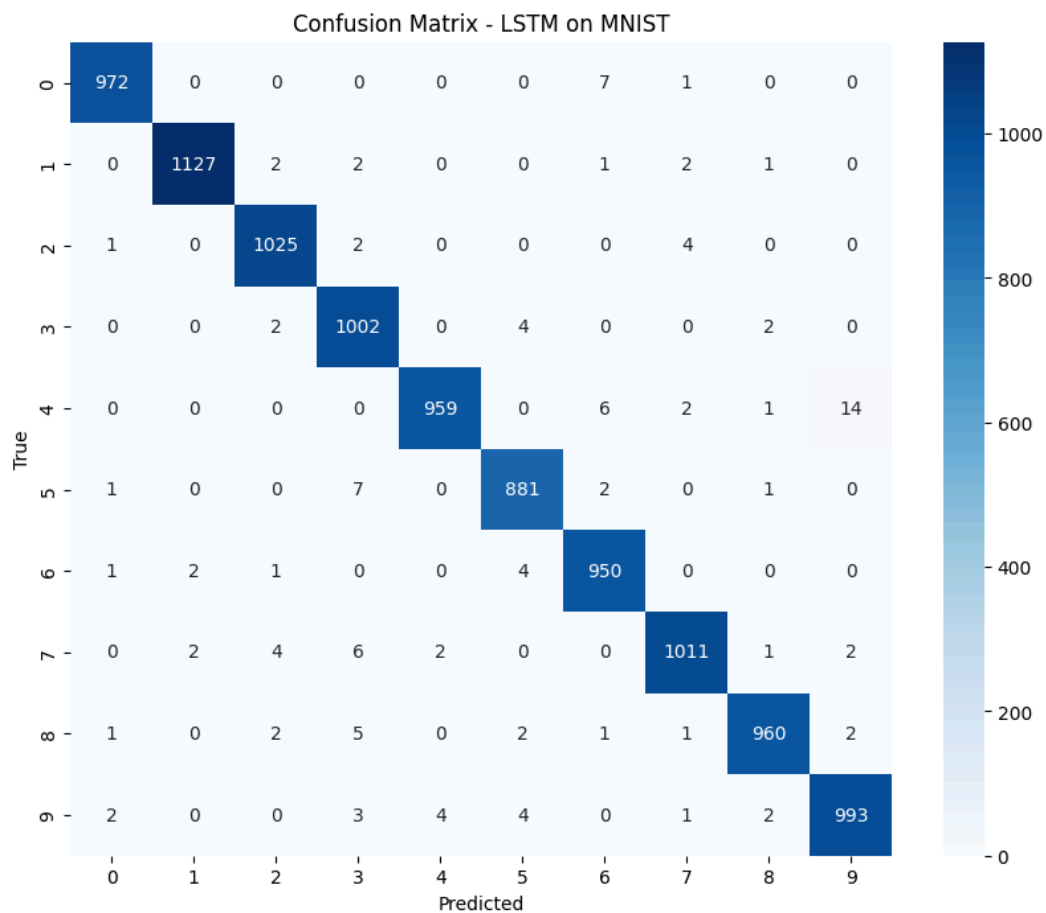
Classification Report:

	precision	recall	f1-score	support
0	0.98	0.97	0.98	980
1	0.99	0.99	0.99	1135
2	0.93	0.95	0.94	1032
3	0.93	0.97	0.95	1010
4	0.97	0.97	0.97	982
5	0.96	0.93	0.94	892
6	0.98	0.97	0.98	958
7	0.96	0.96	0.96	1028
8	0.96	0.98	0.97	974
9	0.96	0.95	0.95	1009
accuracy			0.96	10000
macro avg	0.96	0.96	0.96	10000
weighted avg	0.96	0.96	0.96	10000



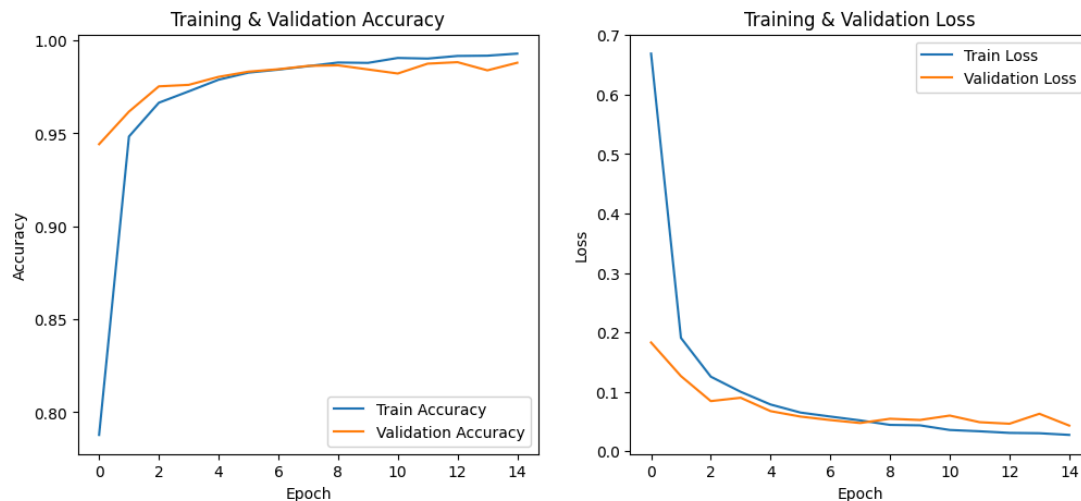
When initialized with ImageNet pretrained weights, VGG-16 achieved slightly lower test accuracy of 96.33%. While still highly accurate, the pretrained filters, optimized for natural images, may have been less suitable for grayscale handwritten digits, which explains the minor drop in performance. This suggests that transfer learning from unrelated domains can sometimes underperform compared to models trained from scratch on domain-specific data.

RNN (LSTM)



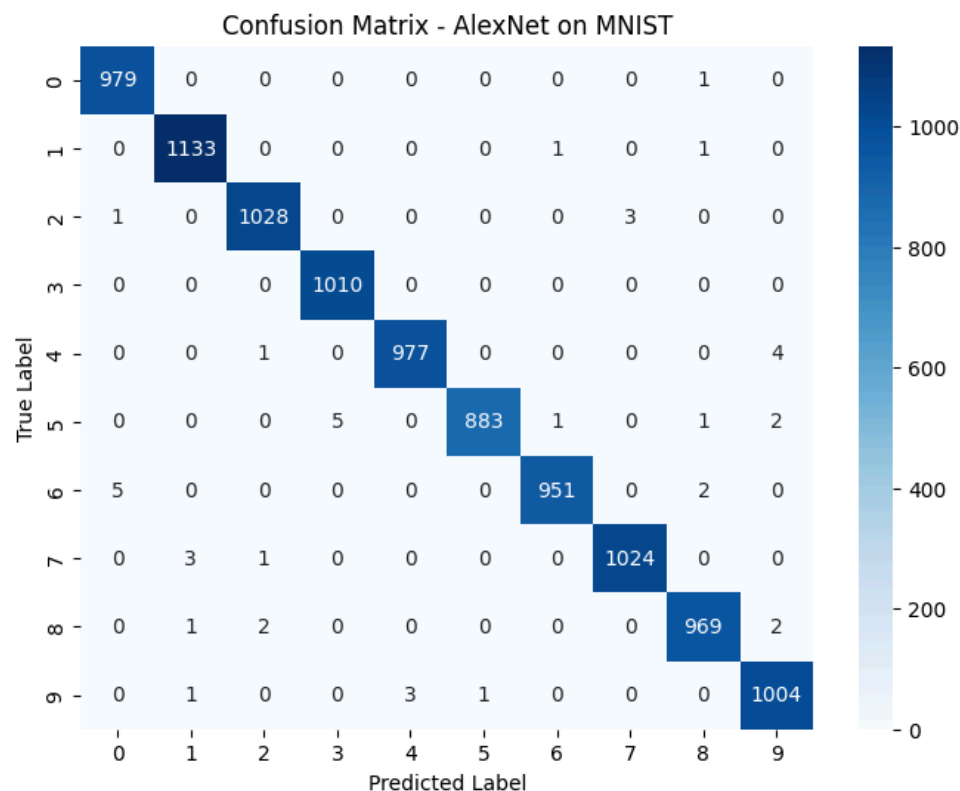
Classification Report:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	980
1	1.00	0.99	0.99	1135
2	0.99	0.99	0.99	1032
3	0.98	0.99	0.98	1010
4	0.99	0.98	0.99	982
5	0.98	0.99	0.99	892
6	0.98	0.99	0.99	958
7	0.99	0.98	0.99	1028
8	0.99	0.99	0.99	974
9	0.98	0.98	0.98	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000



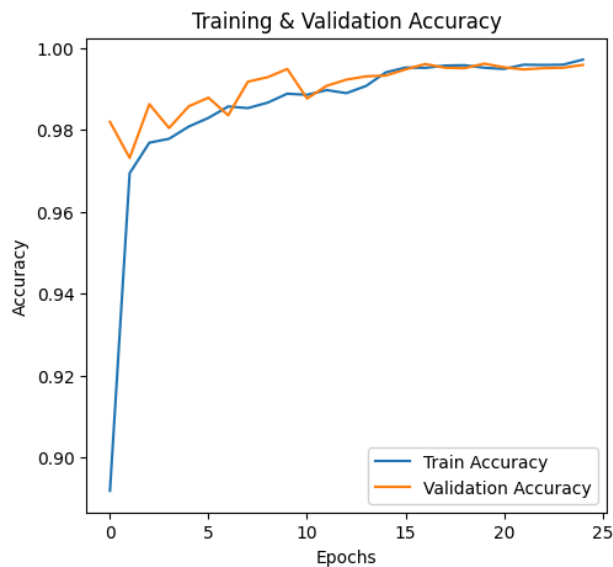
An LSTM-based RNN model also performed well on MNIST, reaching a test accuracy of 98.80%. By treating each image row as a sequence, the LSTM successfully captured spatial dependencies and structural patterns across pixels. The model achieved consistently high precision and recall across all digit classes, indicating robust sequence-based feature learning.

AlexNet



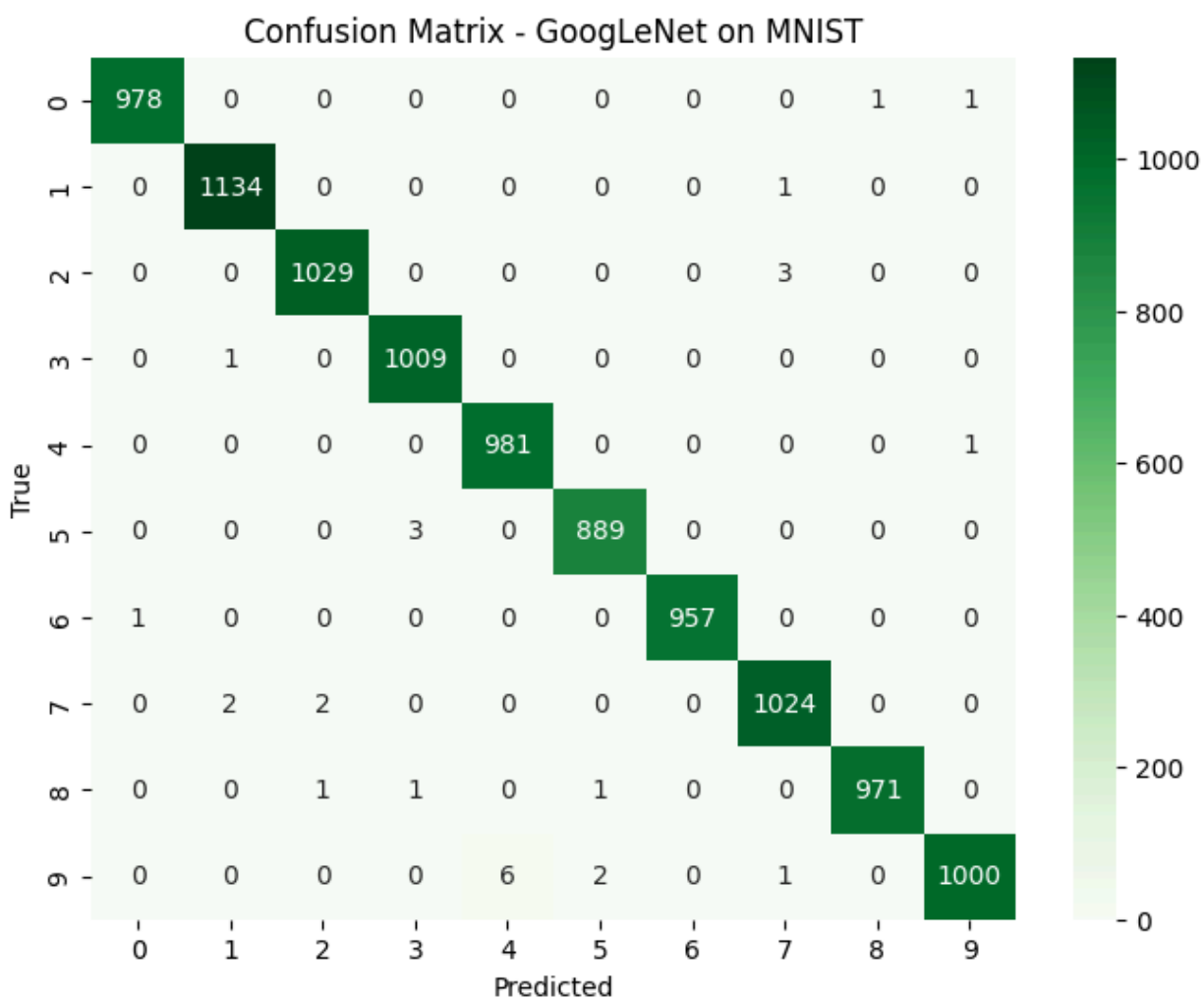
Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	1.00	980
1	1.00	1.00	1.00	1135
2	1.00	1.00	1.00	1032
3	1.00	1.00	1.00	1010
4	1.00	0.99	1.00	982
5	1.00	0.99	0.99	892
6	1.00	0.99	1.00	958
7	1.00	1.00	1.00	1028
8	0.99	0.99	0.99	974
9	0.99	1.00	0.99	1009
accuracy			1.00	10000
macro avg	1.00	1.00	1.00	10000
weighted avg	1.00	1.00	1.00	10000



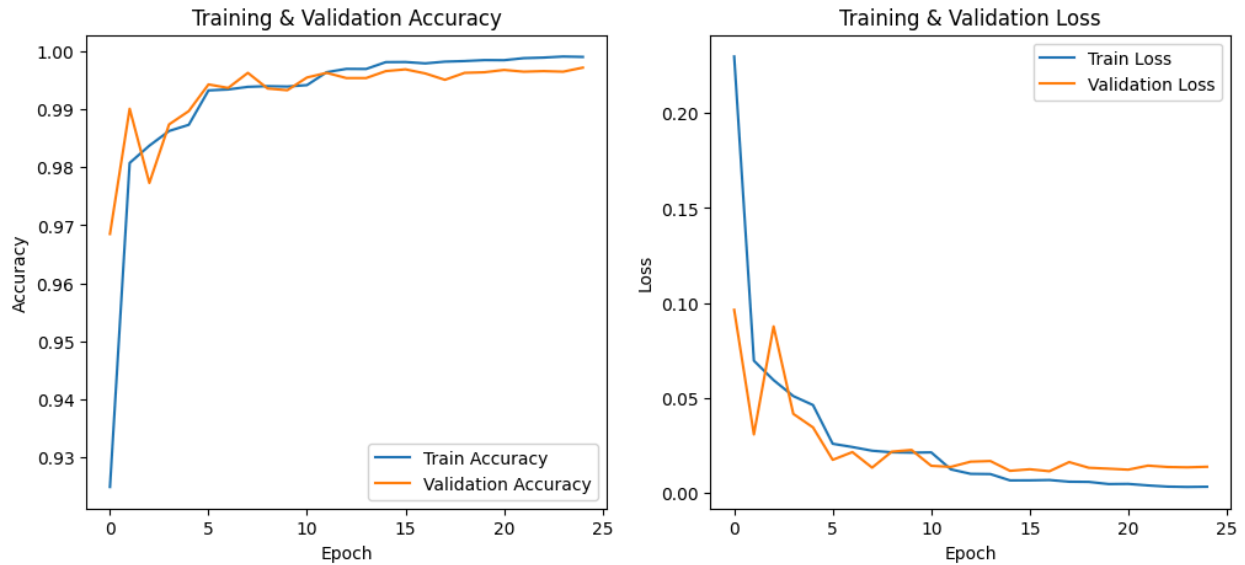
AlexNet delivered excellent performance on MNIST, achieving a near-perfect test accuracy of 99.58%. Its deeper convolutional layers and large receptive fields allowed it to capture hierarchical features effectively, resulting in almost all classes achieving perfect or near-perfect F1-scores. This demonstrates AlexNet's strength in handling even relatively simple image classification tasks when sufficient training is provided.

GoogLeNet (Inception v1)



Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	980
1	1.00	1.00	1.00	1135
2	1.00	1.00	1.00	1032
3	1.00	1.00	1.00	1010
4	0.99	1.00	1.00	982
5	1.00	1.00	1.00	892
6	1.00	1.00	1.00	958
7	1.00	1.00	1.00	1028
8	1.00	1.00	1.00	974
9	1.00	0.99	0.99	1009
accuracy			1.00	10000
macro avg	1.00	1.00	1.00	10000
weighted avg	1.00	1.00	1.00	10000



GoogLeNet achieved the highest test accuracy of 99.72%, outperforming other models slightly. Its inception modules enabled multi-scale feature extraction, efficiently capturing patterns across varying spatial resolutions. The model's classification report shows nearly perfect precision, recall, and F1-scores across all digit classes, highlighting its ability to balance depth and computational efficiency while maintaining exceptional accuracy.

Overall Comparison

Across all architectures, MNIST's simplicity allowed most deep learning models to achieve very high accuracy. VGG-16 trained from scratch outperformed the pretrained variant, indicating that domain-specific training can sometimes be superior to transfer learning. AlexNet and GoogLeNet excelled due to their hierarchical and multi-scale feature extraction capabilities, with GoogLeNet marginally surpassing others. LSTM models performed competitively, demonstrating the effectiveness of sequence-based learning for image data. Overall, MNIST confirms that even moderate-sized convolutional and sequence-based networks can achieve near-perfect classification on well-structured datasets.